Theory Notes / version 1.0

Machine Learning A

Casper Moes (hwd362) 5. November 2026

Contents

Generalization Bounds **8**

1 Concentration of measure inequalities **9**

1.1 Markow's inequality 9

1.2 Chebychew's inequality 10

1.3 Hoeffding's inequality 10

2 Generalization bounds: Finite and Countably Infinity Hypothesis Classes **12**

2.1 Single hypothesis 12

2.1.1 Setting and risk definitions 12

2.1.2 Theoretical properties of empirical risk 13

2.1.3 Sidenote: Understanding $\epsilon = \sqrt{\frac{1}{2N} \ln\left(\frac{1}{\delta}\right)}$ 14

2.2 Finite Hypothesis class 15

2.2.1 Hoeffdings bound does not apply anymore! 16

2.3 Lower Bound 18

2.4 Approximation and Estimation error 19

Supervised Learning **20**

3 Linear regression **21**

3.1 Linear Algebra prerequisites 21

3.1.1 Singular Value Decomposition 21

3.1.2 The Spectral Theorem 22

3.2 Overidentified case: $N > rank(X)$ 24

3.2.1 OLS is equivalent with Gaussian MLE 26

3.3 Underidentified case: $N < rank(X)$ 26

3.3.1 Minimum-norm solution 27

3.3.2 Singular Value Decomposition and the Pseudoinverse . . 28

3.3.3 Ridge regression (Two-norm regularization) 29

3.3.4 Equivalence theorem 29

3.3.5 Lasso regression (One-norm regularization) 31

3.4 Comment on double descent 31

4 Linear Classification **32**

4.1 Logistic regression 32

4.1.1 Binary case 32

4.1.2 Multiple classification 35

4.1.3 Optimization: Gradient descent methods 36

5 Random Forest **37**

5.1 Binary decision trees (definitions) 37

5.1.1 Regression trees 38

5.1.2 Classification trees . . . 38
5.1.3 Growing and pruning . . . 39
5.2 Ensembles and bagging . . . 39
5.3 Growing and evaluating a random forest . . . 40
5.4 Out-of-bag (OOB) error . . . 40
5.5 Discussion: CART vs Random Forest . . . 41

6 Neural Networks **41**
6.1 The Neuron . . . 41
6.1.1 Pre-Activation . . . 42
6.1.2 Activation and output . . . 42
6.1.3 Activation functions . . . 43
6.2 Gradient descent . . . 46
6.3 Backpropagation . . . 47
6.3.1 The ouput layer . . . 48
6.3.2 Hidden layers . . . 49

7 Deep Learning **49**
7.1 Convolution / cross-correlation . . . 49
7.1.1 Padding . . . 51
7.1.2 Equivariance . . . 51
7.2 Pooling layer . . . 51
7.2.1 Invariance . . . 52
7.2.2 Receptive field . . . 52

7.3 Example: VGG-16 architecture . . . 53
7.4 Tricks-of-the-Trade . . . 54
7.4.1 Ensemble Methods . . . 54
7.4.2 Dropout . . . 54
7.4.3 Residual Leraning / Shorcuts . . . 55
7.4.4 Augmentation og Invariance . . . 55
7.4.5 Batch and Layer Normalization . . . 56

Unsupervised Learning **57**

8 Clustering **58**
8.1 General definitions . . . 58
8.1.1 Mathematical setting . . . 58
8.2 The k-means problem . . . 59
8.2.1 The objective function . . . 59
8.2.2 Special case $k = 1$ and why the mean appears . . . 60
8.3 The k-means algorithm (Lloyd's algorithm) . . . 61
8.3.1 Alternating between assignments and centers . . . 61
8.3.2 Why the algorithm converges . . . 62
8.3.3 A four-point example with arbitrarily bad cost . . . 62
8.3.4 Class exercise with three rows and three columns . . . 63
8.4 Choosing the initial centers . . . 64
8.4.1 k-means++ . . . 64

8.4.2 Worked example of *k*-means++ . . . 65
8.4.3 Guarantee in expectation . . . 66
8.5 Choosing the number of clusters . . . 66
8.6 Beyond *k*-means . . . 67
8.6.1 *k*-medoids and the choice of representative . . . 67
8.6.2 Outliers and why we might prefer the median . . . 67
8.6.3 Why the median minimizes absolute deviations . . . 68
8.6.4 The *k*-medians problem and algorithm . . . 69
8.7 Flat versus hierarchical clustering . . . 69
8.8 Conceptual issues . . . 70
8.8.1 Similarity depends on the question . . . 70
8.8.2 Cluster membership is transitive . . . 70

9 Principal Component Analysis (linear Dimensionality Reduction) 71

9.1 Definitions . . . 71
9.1.1 Dimensions . . . 71
9.2 The objective of PCA . . . 71
9.3 Optimal reconstruction . . . 72
9.3.1 Changing basis and keeping fewer directions . . . 72
9.3.2 Why we center the data . . . 74
9.4 Choosing the principal directions . . . 74
9.4.1 Minimizing reconstruction error is maximizing variance . 74
9.4.2 Why we take the eigenvectors with the largest eigenvalues 75

9.5 Computing PCA . . . 77
9.5.1 Using the covariance matrix . . . 77
9.5.2 Using the singular value decomposition . . . 77
9.5.3 Connection to low-rank approximation . . . 78
9.6 Explained variance and choosing the dimension . . . 78
9.6.1 Worked example in two dimensions . . . 79
9.6.2 Example with faces . . . 80
9.7 PCA and preprocessing . . . 80
9.7.1 Centering and scaling . . . 80
9.7.2 PCA whitening and ZCA . . . 81
9.8 When does PCA work well? . . . 81

10 Stochastic Neighborhood Embedding (non-linear Dimensionality Reduction) 82

10.1 The task of dimensionality reduction . . . 82
10.1.1 Why might a linear projection be a problem? . . . 83
10.2 Symmetric Stochastic Neighbor Embedding (SNE) . . . 83
10.2.1 Affinities in the original data X . . . 84
10.2.2 How do we choose σ_i? . . . 84
10.2.3 Affinities in the embedding Y and the objective . . . 85
10.3 t-SNE . . . 86
10.3.1 The crowding problem . . . 86
10.3.2 What changes in t-SNE? . . . 87
10.4 What do the plots in the slides show? . . . 87

10.4.1 MNIST 87
10.4.2 Gaussian mixture, Swiss roll and spiral 88
10.4.3 The home assignment example 88
10.5 How to read a t-SNE plot 88

I | Generalization Bounds

Concentration inequalities, empirical risk,
and bounds for finite and countable hypothesis classes

1 Concentration of measure inequalities

1.1 Markow's inequality

Statement: For any non-negative stochastic variable X and with $\epsilon > 0$ we have

$$\mathbb{P}(X \geq \epsilon) \leq \frac{\mathbb{E}[X]}{\epsilon}.$$

Proof: Define $Y := 1(X \geq \epsilon)$ then $Y \leq \frac{X}{\epsilon}$ (if $0 \leq X < \epsilon$ then $Y = 0 \leq \frac{X}{\epsilon}$ and if $X \geq \epsilon \iff \frac{X}{\epsilon} \geq 1$ then $Y = 1 \leq \frac{X}{\epsilon}$) therefore $\mathbb{P}(X \geq \epsilon) = \mathbb{P}(Y = 1) = \mathbb{E}[Y] \leq \mathbb{E}[\frac{X}{\epsilon}] = \frac{\mathbb{E}[X]}{\epsilon}$.

This can be rewritten by defining $a = \frac{\epsilon}{\mathbb{E}[X]}$ ($\mathbb{E}[X] \neq 0$) and we have

$$\mathbb{P}(X \geq a\mathbb{E}[X]) \leq \frac{1}{a}.$$

That is, the probability of an event a times away from the stochastic variables mean value is decaying with $1/a$.

Another (simpler) proof:

$$\begin{aligned}
\mathbb{E}[X] &= \mathbb{E}\left[X\mathbb{I}_{(X\geq\epsilon)} + X\mathbb{I}_{(X<\epsilon)}\right] \\
&= \mathbb{E}\left[X\mathbb{I}_{(X\geq\epsilon)}\right] + \underbrace{\mathbb{E}\left[X\mathbb{I}_{(X<\epsilon)}\right]}_{\geq 0 \text{ (X is non negative)}} \\
&\geq \mathbb{E}\left[X\mathbb{I}_{(X\geq\epsilon)}\right] \\
&= \int_{\epsilon}^{\infty} x\mathbb{P}(X = x)\, dx \qquad \left(\sum_{x\geq\epsilon} x\mathbb{P}(X = x)\right) \\
&\geq \int_{\epsilon}^{\infty} \epsilon\mathbb{P}(X = x)\, dx \qquad \left(\sum_{x\geq\epsilon} \epsilon\mathbb{P}(X = x)\right) \\
&= \epsilon\mathbb{P}(X \geq \epsilon).
\end{aligned}$$

Rearranging yields Markow's inequality

$$\mathbb{P}\left(X \geq \epsilon\right) \geq \frac{\mathbb{E}\left[X\right]}{\epsilon}.$$

1.2 Chebychew's inequality

Statement:

$$\mathbb{P}(|X - \mathbb{E}[X]| \geq \epsilon) \leq \frac{\mathbb{V}(X)}{\epsilon^2}.$$

Proof: For any $\epsilon > 0$ we have (by Markow's inequality)

$$\mathbb{P}((X - \mathbb{E}[X])^2 \geq \epsilon^2) \leq \frac{\mathbb{E}[(X - \mathbb{E}[X])^2]}{\epsilon^2} = \frac{\mathbb{V}(X)}{\epsilon^2} -$$

Finally we note that

$$|X - \mathbb{E}[X]| \geq \epsilon \iff (X - \mathbb{E}[X])^2 \geq \epsilon^2$$

and thus $\mathbb{P}(|X - \mathbb{E}[X]| \geq \epsilon) \leq \frac{\mathbb{V}(X)}{\epsilon^2}$.

1.3 Hoeffding's inequality

Statement: Given the independent stochastic variables $X_1, \ldots, X_n$ where there exists $a_{i,} b_i$ for $i \in \{1, \ldots, n\}$ such that $X_i \in [a_i, b_i]$. For all $\epsilon > 0$,

$$\mathbb{P}\left(\sum_{i=1}^{n} X_i - \mathbb{E}\left[\sum_{i=1}^{n} X_i\right] \geq \epsilon\right) \leq \exp\left\{-\frac{2\epsilon^2}{\sum_{i=1}^{n}(b_i - a_i)^2}\right\}.$$

Proof: Start by simple rewriting and application of Markow's inequality - which is legal since $S_X \equiv \sum_{i=1}^{n} X_i$ are composed of i.i.d. stochastic variables and thus $S_X - \texttt{constant}$ itself is a stochastic variable (here $\texttt{constant} = \mathbb{E}\left(\sum_{i=1}^{n} X_i\right)$

$$\begin{aligned}\mathbb{P}\left(\sum_{i=1}^{n} X_i - \mathbb{E}\left[\sum_{i=1}^{n} X_i\right] \geq \epsilon\right) &= \mathbb{P}\left(e^{\lambda(\sum_{i=1}^{n} X_i - \mathbb{E}[\sum_{i=1}^{n} X_i])} \geq e^{\lambda\epsilon}\right) \\ &\leq \frac{E\left[e^{\lambda(\sum_{i=1}^{n} X_i - \mathbb{E}[\sum_{i=1}^{n} X_i])}\right]}{e^{\lambda\epsilon}}.\end{aligned}$$

The nominator can be reduced. Firstly we use that the expectation operator is linear so that $\mathbb{E}\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n \mathbb{E}[X_i]$

$$\begin{aligned}\mathbb{E}\left[e^{\lambda(\sum_{i=1}^n X_i - \mathbb{E}[\sum_{i=1}^n X_i])}\right] &= \mathbb{E}\left[e^{\lambda(\sum_{i=1}^n X_i - \sum_{i=1}^n \mathbb{E}[X_i])}\right] \\ &= \mathbb{E}\left[e^{\lambda \sum_{i=1}^n (X_i - \mathbb{E}[X_i])}\right].\end{aligned}$$

Now using the properties of the exponential function and Hoeffdings lemma

$$\begin{aligned}\mathbb{E}\left[e^{\lambda \sum_{i=1}^n X_i - \mathbb{E}[X_i]}\right] &= \mathbb{E}\left[\prod_{i=1}^n e^{\lambda(X_i - \mathbb{E}[X_i])}\right] \\ &\overset{indep.}{=} \prod_{i=1}^n E\left[e^{\lambda(X_i - \mathbb{E}[X_i])}\right] \\ &\leq \prod_{i=1}^n e^{\lambda^2 (b_i - a_i)^2/8} \\ &= e^{\lambda^2/8 \sum_{i=1}^n (b_i - a_i)^2}.\end{aligned}$$

Taking stock: so far we have

$$\mathbb{P}\left(\sum_{i=1}^n X_i - \mathbb{E}\left[\sum_{i=1}^n X_i\right] \geq \epsilon\right) \leq e^{\lambda^2/8 \sum_{i=1}^n (b_i - a_i)^2 - \lambda\epsilon}.$$

However we want the tightest probability bound, and therefore we minimize the right hand side expression wrt. $\lambda > 0$.

$$\lambda^* = \arg\min_{\lambda \in \mathbb{R}_+} \; e^{\lambda^2/8 \sum_{i=1}^n (b_i - a_i)^2 - \lambda\epsilon} = \arg\min_{\lambda \in \mathbb{R}_+} \left\{\frac{\lambda^2}{8}\sum_{i=1}^n (b_i - a_i)^2 - \lambda\epsilon\right\},$$

with necessary first order condition

$$0 = \frac{\lambda^*}{4}\sum_{i=1}^n (b_i - a_i)^2 - \epsilon \iff \lambda^* = \frac{4\epsilon}{\sum_{i=1}^n (b_i - a_i)^2}.$$

Substituting this into the stock-taking expression yields the theorem statement

$$
\begin{aligned}
e^{(\lambda^*)^2/8\sum_{i=1}^n (b_i-a_i)^2-\lambda^*\epsilon} &= \exp\left\{\left(\frac{4\epsilon}{\sum_{i=1}^n (b_i-a_i)^2}\cdot\right)^2 \frac{1}{8}\sum_{i=1}^n (b_i-a_i)^2 - \frac{4\epsilon^2}{\sum_{i=1}^n (b_i-a_i)^2}\right\} \\
&= \exp\left\{\frac{2\epsilon^2}{\sum_{i=1}^n (b_i-a_i)^2} - \frac{4\epsilon^2}{\sum_{i=1}^n (b_i-a_i)^2}\right\} \\
&= \exp\left\{-\frac{2\epsilon^2}{\sum_{i=1}^n (b_i-a_i)^2}\right\}.
\end{aligned}
$$

Taken all together we have obtained Hoeffding Inequality:

$$
\mathbb{P}\left(\sum_{i=1}^n X_i - \mathbb{E}\left[\sum_{i=1}^n X_i\right] \geq \epsilon\right) \leq \exp\left\{-\frac{2\epsilon^2}{\sum_{i=1}^n (b_i-a_i)^2}\right\}.
$$

2 Generalization bounds: Finite and Countably Infinity Hypothesis Classes

2.1 Single hypothesis

2.1.1 Setting and risk definitions

Assume the loss function $\ell(h(X), Y)$ is bounded on $[0, 1]$. This is important for the specific calculations but the general insights are applicable to any loss function bounds $[a, b]$.

True risk: The quality of any hypothesis h is measured by its (true) risk

$$
L(h) \equiv \mathbb{E}_{(X,Y)\sim p}\left[\ell(h(X), Y)\right],
$$

where p is the *unknown* distribution over which the world has generated (X, Y) - i.e. pairs of features from the input space $\mathcal{X}$ and outcomes from the label space $\mathcal{Y}$.

Empirical risk: Using a validation sample

$$
S = \{(X_1, Y_1), \ldots, (X_N, Y_N)\}, \quad (X_i, Y_i) \overset{iid}{\sim} p,
$$

we calculate the empirical risk defined by

$$\hat{L}(h, S) \equiv \frac{1}{N} \sum_{n=1}^{N} \ell(h(X_n), Y_n).$$

2.1.2 Theoretical properties of empirical risk

Unbiased: Since the validation samples are independently and identically distributed from the distribution p we have

$$\begin{aligned}
\mathbb{E}_{S \sim p^N}\left[\hat{L}(h, S)\right] &= \mathbb{E}_{S \sim p^N}\left[\frac{1}{N} \sum_{n=1}^{N} \ell(h(X_n), Y_n)\right] \\
&= \frac{1}{N} \sum_{n=1}^{N} \mathbb{E}_{(X_n, Y_n) \sim p}\left[\ell(h(X_n), Y_n)\right] \\
&= \frac{1}{N} \sum_{n=1}^{N} \mathbb{E}_{(X_1, Y_1) \sim p}\left[\ell(h(X_1), Y_1)\right] \\
&= \mathbb{E}_{(X_1, Y_1) \sim p}\left[\ell(h(X_1), Y_1)\right] \\
&= L(h).
\end{aligned}$$

However the above only holds because h is independent of S. If not we generally have

$$\mathbb{E}_{S \sim p^N}\left[\hat{L}\left(\hat{h}_S, S\right)\right] \neq \mathbb{E}_{S \sim p^N}\left[L\left(\hat{h}_S\right)\right]$$

What can be said about the distribution of $\hat{L}(h, S)$? Formally what is δ here

$$\mathbb{P}\left(L(h) - \hat{L}(h, S) \geq \epsilon\right) \leq \delta.$$

I.e. with what tightness can we bound the probability that the true risk L is $\epsilon > 0$ bigger than $\hat{L}$. For example we might want a confindence of $\delta = 0.05$ that is w.p. less than 0.05 the true loss L is $\epsilon = f(\delta)$ - that depends on δ units bigger than $\hat{L}$.

Applying Hoeffdings bound yields

$$\begin{aligned}
\mathbb{P}\left(L(h) - \hat{L}(h,S) \geq \epsilon\right) &= \mathbb{P}\left(\underbrace{\mathbb{E}\left[\frac{1}{N}\sum_{n=1}^{N}\ell(h(X_n),Y_n)\right]}_{=\mathbb{E}[\ell(h(X_1),Y_1)]=L(h)} - \frac{1}{N}\sum_{n=1}^{N}\ell(h(X_n),Y_n) \geq \epsilon\right) \\
&= \mathbb{P}\left(\mathbb{E}\left[\sum_{n=1}^{N}\ell(h(X_n),Y_n)\right] - \sum_{n=1}^{N}\ell(h(X_n),Y_n) \geq N\epsilon\right) \\
&\leq \exp\left\{-\frac{2(N\epsilon)^2}{\sum_{n=1}^{N}(b_n-a_n)^2}\right\} \\
&= \exp\left\{-2N\epsilon^2\right\}.
\end{aligned}$$

Defining the relation $\delta = \exp\left\{-2N\epsilon^2\right\}$ yields $\epsilon = \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}$. Thus

$$\mathbb{P}\left(L(h) \geq \hat{L}(h,S) + \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}\right) \leq \delta,$$

and taking the complement

$$\mathbb{P}\left(L(h) < \hat{L}(h,S) + \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}\right) = 1 - \mathbb{P}\left(L(h) \geq \hat{L}(h,S) + \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}\right) \geq 1-\delta.$$

2.1.3 *Sidenote: Understanding* $\epsilon = \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}$

Earlier we obtained

$$\mathbb{P}\left(L(h) - \hat{L}(h,S) \geq \epsilon\right) \leq \exp\left\{-2N\epsilon^2\right\}.$$

we can think if this inequality as asking: *If you set an error-margin* $\epsilon > 0$ *then I tell you how confident I am that an error happens (i.e.* $L - \hat{L} \geq \epsilon$*).* If we instead ask the question: *I want the probability that an error happens* (*i.e.*$L - \hat{L} \geq \epsilon$) *to be* $\delta \cdot 100\%$ *how large does the error-margin* ϵ *have to be?* The latter case is answered by

relabeling the right hand side as $\delta = \exp\left\{-2N\epsilon^2\right\}$ and then backing out what ϵ must be: $\epsilon = \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}$.

The complement flips the question:

$$\mathbb{P}\left(L(h) < \hat{L}(h,S) + \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}\right) \geq 1-\delta,$$

and asks: The probability that L is acceptable (i.e. smaller than $\hat{L}(h,S) + \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}$) is at least $1-\delta$. Or: I am $(1-\delta)\cdot 100\%$ certain that L is smaller than $\hat{L}(h,S) + \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}$.

2.2 Finite Hypothesis class

Unlike before we have now trained $h_1, \ldots, h_M$ hypotheses as such

$$\begin{aligned}
h_1 &\longrightarrow \hat{L}(h_1,S) = \frac{1}{N}\sum_{n=1}^{N}\ell(h_1(X_n),Y_n)\\
h_2 &\longrightarrow \hat{L}(h_2,S) = \frac{1}{N}\sum_{n=1}^{N}\ell(h_2(X_n),Y_n)\\
&\vdots\\
h_M &\longrightarrow \hat{L}(h_M,S) = \frac{1}{N}\sum_{n=1}^{N}\ell(h_M(X_n),Y_n),
\end{aligned}$$

and chosen $\hat{h}_S^*$ with empirical risk minimization

$$\begin{aligned}
\hat{h}_S^* &= \arg\min_{h\in\mathcal{H}}\hat{L}(h,S)\\
&= \arg\min_{h\in\mathcal{H}}\left\{\frac{1}{N}\sum_{n=1}^{N}\ell(h(X_n),Y_n)\right\},
\end{aligned}$$

where S is a validation sample (i.e. not used for training) with elements

$(X_n, Y_n) \overset{iid}{\sim} p$.

2.2.1 *Hoeffdings bound does not apply anymore!*

The question is as follows: *How bad can we expect our model to perform?* This is measured in terms of (true) risk

$$\mathbb{E}_{(X,Y)\sim p}\left[\ell\left(\hat{h}_S^*(X), Y\right)\right],$$

where p is the probability distribution of the stochastic vector (X, Y). We would ideally just plug and play with Hoeffding's inequality. However this requires that

$$L\left(\hat{h}_S^*\right) = \mathbb{E}_S\left[\hat{L}\left(\hat{h}_S^*, S\right)\right],$$

since

$$\begin{aligned}\mathbb{P}\left(L\left(\hat{h}_S^*\right) - \hat{L}\left(\hat{h}_S^*, S\right) \geq \epsilon\right) &= \mathbb{P}\left(L\left(\hat{h}_S^*\right) - \frac{1}{N}\sum_{n=1}^{N}\ell\left(\hat{h}_S^*(X_n), Y_n\right) \geq \epsilon\right) \\ &= \mathbb{P}\left(NL\left(\hat{h}_S^*\right) - \sum_{n=1}^{N}\ell\left(\hat{h}_S^*(X_n), Y_n\right) \geq N\epsilon\right)\end{aligned}$$

For Hoeffdings bound to apply we would therefore need that[1]

$$NL\left(\hat{h}_S^*\right) = \mathbb{E}_{S\sim p^N}\left[\sum_{n=1}^{N}\ell\left(\hat{h}_S^*(X_n), Y_n\right)\right]$$

[1]To see this simply look at the inequality

$$\mathbb{P}\left(\mathbb{E}\left[\sum_{n=1}^{N} X_n\right] - \sum_{n=1}^{N} X_n \geq \epsilon\right) \leq \exp\left\{-\frac{2\epsilon^2}{\sum_{n=1}^{N}(b_n - a_n)^2}\right\},$$

from where we see that the left term needs to be $\mathbb{E}\left[\sum_{n=1}^{N} X_n\right]$ i.e. the expectation of the right term $\sum_{n=1}^{N} X_n$.

which implies

$$L\left(\hat{h}^*_S\right) = \mathbb{E}_{S\sim p^N}\left[\frac{1}{N}\sum_{n=1}^{N}\ell\left(\hat{h}^*_S(X_n), Y_n\right)\right] = \mathbb{E}_{S\sim p^N}\left[\hat{L}\left(\hat{h}^*_S, S\right)\right].$$

This however does not generally hold since

$$\begin{aligned}\mathbb{E}_{S\sim p^N}\left[\hat{L}\left(\hat{h}^*_S, S\right)\right] &= \mathbb{E}_{S\sim p^N}\left[\frac{1}{N}\sum_{n=1}^{N}\ell\left(\hat{h}^*_S(X_n), Y_n\right)\right] \\ &\neq \mathbb{E}_{(X_1,Y_1)\sim p}\left[\frac{1}{N}\sum_{n=1}^{N}\ell\left(\hat{h}^*_S(X_1), Y_1\right)\right],\end{aligned}$$

since $\hat{h^*}_S$ generally depends on the whole sample $S = \{(X_1, Y_1), \ldots, (X_N, Y_N)\}$. Therefore the stochastic variables $\ell\left(\hat{h^*}_S\left(X_n\right), Y_n\right)$ are not independent (because $\hat{h^*}_S$ was selected using $S_{-(X_n,Y_n)}$) and therefore Hoeffding's inequality does not apply.

The solution: Therefore we somehow need to break dependence of the hypothesis $\hat{h^*}_S$ on the sample:

$$\begin{aligned}\mathbb{P}\left(L\left(\hat{h}^*_S\right) - \hat{L}\left(\hat{h}^*_S, S\right) \geq \epsilon\right) &\leq \mathbb{P}\left(L\left(h_1\right) - \hat{L}\left(h_1, S\right) \geq \epsilon \text{ or } \cdots \text{ or } L\left(h_M\right) - \hat{L}\left(h_M, S\right) \geq \epsilon\right) \\ &= \mathbb{P}\left(\exists h \in \mathcal{H} : L\left(h\right) - \hat{L}\left(h, S\right) \geq \epsilon\right) \\ &\leq \sum_{j=1}^{M}\mathbb{P}\left(L\left(h_j\right) - \hat{L}\left(h_j, S\right) \geq \epsilon\right) \\ &= \sum_{j=1}^{M}\mathbb{P}\left(N\underbrace{L\left(h_j\right)}_{\equiv\mathbb{E}_{(X,Y)\sim p}\left[\ell\left(h_j(X),Y\right)\right]} - \sum_{n=1}^{N}\ell\left(h_j(X_n), Y_n\right) \geq N\epsilon\right) \\ &= \sum_{j=1}^{M}\mathbb{P}\left(N\mathbb{E}_{S\sim p^N}\left[\frac{1}{N}\sum_{n=1}^{N}\ell\left(h_j(X), Y\right)\right] - \sum_{n=1}^{N}\ell\left(h_j(X_n), Y_n\right) \geq N\epsilon\right) \\ &\leq M\exp\left\{-\frac{2(N\epsilon)^2}{\sum_{n=1}^{N}(b_n - a_n)^2}\right\} \\ &= M\exp\left\{-2N\epsilon^2\right\},\end{aligned}$$

The first inequality holds simply because $\hat{h}_S^* \in \{h_1, h_2, \ldots, h_M\}$. The second inequality holds by the union bound:

$$\mathbb{P}(A \cup B) = P(A) + P(B) - \mathbb{P}(A \cap B) \leq P(A) + P(B).$$

From this we obtain

$$\delta = M \exp\left\{-2N\epsilon^2\right\} \iff \epsilon = \sqrt{\frac{1}{2N} \ln\left(\frac{M}{\delta}\right)},$$

and further

$$\mathbb{P}\left(L\left(\hat{h}_S^*\right) \geq \hat{L}\left(\hat{h}_S^*, S\right) + \sqrt{\frac{1}{2N} \ln\left(\frac{M}{\delta}\right)}\right) \leq \delta$$
$$\mathbb{P}\left(L\left(\hat{h}_S^*\right) < \hat{L}\left(\hat{h}_S^*, S\right) + \sqrt{\frac{1}{2N} \ln\left(\frac{M}{\delta}\right)}\right) \geq 1 - \delta.$$

This is the result: From a finite predetermined hypothesis class with cardinality $|\mathcal{H}| = M$ using the Empirical Risk Minimizer (ERM) - or any selection over h! Since:

$$\mathbb{P}\left(\exists h \in \mathcal{H} : L(h) \geq \hat{L}(h, S) + \sqrt{\frac{1}{2N} \ln\left(\frac{M}{\delta}\right)}\right) \leq \delta,$$

and in particular for the ERM hypothesis $\hat{h}_S^*$

$$\mathbb{P}\left(L\left(\hat{h}_S^*\right) \geq \hat{L}\left(\hat{h}_S^*, S\right) + \sqrt{\frac{1}{2N} \ln\left(\frac{M}{\delta}\right)}\right) \leq \delta.$$

2.3 Lower Bound

In general there is a lower bound on how well bounding inequalities can work. Concretely we use Hoeffding's inequality, but could this be improved? And how much? To answer the first: yes, it has been improved. For the second question: we can show that there is a limit to how much bounding can happen in general. We call this a lower bound which answers the question: In general what is the

lowest upper bound we can impose on an event? or: It tells us that we can never be *this* certain (i.e. δ cannot be this low) using bounding techniques.

In short the lectures construct an example for a specific hypothesis space $\mathcal{H}$ where

$$\mathbb{P}\left(\exists h \in \mathcal{H} : L\left(h\right) \geq \hat{L}\left(h, S\right) + \frac{1}{8}\right) \geq \frac{1}{8}.$$

2.4 Approximation and Estimation error

Approximation error comes from restricting $\mathcal{H}$ so that the Bayes optimal hypothestical choice is omittet (i.e. the h that would generate the lowest $L(h)$ is not in $\mathcal{H}$).

Estimation defines the variance in selecting the best hypothesis in class $\mathcal{H}$: there is an $h^* = \arg\min_{h \in \mathcal{H}} L(h)$ but using data we are not sure to find it. we most likely find $L(\hat{h}) < L(h^*)$. This is the estimation error.

II | Supervised Learning

Linear regression, logistic regression,
decision trees, random forests, and neural networks

3 Linear regression

3.1 Linear Algebra prerequisites

3.1.1 Singular Value Decomposition

Statement: For any real matrix $X \in \mathbb{R}^{N \times d}$ we can write

$$X = U\Gamma V',$$

where U and V are semi-orthogonal and Γ is a diagonal matrix with non-negative entries. The dimensions of all matrices depend on $r = rank(X)$ see proof.

Proof: Consider the symmetric matrix $B = X'X \in \mathbb{R}^{n \times n}$ (which can be shown in one line: $(X'X)' = X'X$) that by the spectral theorem B has n orthonormal eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_n$. Furthermore this implies that the eigenvalues are non-negative:

$$\begin{aligned}
\lambda_i &= \lambda_i \mathbf{v}_i' \mathbf{v}_i \\
&= \mathbf{v}_i' \lambda_i \mathbf{v}_i \\
&= \mathbf{v}_i' (X'X) \mathbf{v}_i \\
&= (X\mathbf{v}_i)'(X\mathbf{v}_i) \\
&= ||X\mathbf{v}_i||_2^2 \geq 0.
\end{aligned}$$

Now define the singular values $\sigma_i \equiv \sqrt{\lambda_i}$ and define $\mathbf{u}_i \equiv X\mathbf{v}_i/\sigma_i$. The latter can be shown to be orthonormal for the collection $i = 1, \ldots, r$ where $r = rank(X)$.

To verify orthonormality, note that for $i, j \leq r$,

$$\begin{aligned}
\mathbf{u}_i' \mathbf{u}_j &= \frac{\mathbf{v}_i' X' X \mathbf{v}_j}{\sigma_i \sigma_j} \\
&= \frac{\lambda_j}{\sigma_i \sigma_j} \mathbf{v}_i' \mathbf{v}_j \\
&= \frac{\sigma_j}{\sigma_i} \delta_{ij} = \delta_{ij}.
\end{aligned}$$

We can complete these r vectors to an orthonormal basis of $\mathbb{R}^N$, forming the orthogonal matrix $U = [\mathbf{u}_1, \ldots, \mathbf{u}_N]$. Similarly, let $V = [\mathbf{v}_1, \ldots, \mathbf{v}_d]$.

By construction, $X\mathbf{v}_i = \sigma_i \mathbf{u}_i$ for $i \leq r$. For $i > r$, we have $\lambda_i = 0$, implying $\|X\mathbf{v}_i\|_2^2 = 0$ and hence $X\mathbf{v}_i = 0$.

Consequently,

$$XV = U\Gamma,$$

where $\Gamma \in \mathbb{R}^{N \times d}$ is rectangular diagonal with entries $\sigma_1, \ldots, \sigma_r > 0$ and zeros elsewhere. Since $VV' = I_d$,

$$X = U\Gamma V'.$$

This holds regardless of whether $N > d$ or $N < d$.

3.1.2 The Spectral Theorem

Statement: Any real valued symmetrix matrix $A \in \mathbb{R}^{n \times n}$ can be written as

$$A = Q\Lambda Q',$$

where $Q = \begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_n \end{bmatrix} \in \mathbb{R}^{n \times n}$ holds n orthogonal real-valued eigenvectors and $\Lambda = diag(\lambda_1, \cdots, \lambda_n)$ holds their respective eigenvalues.

Proof: Firstly we show that any real valued symmetric (i.e. $A = A'$) matrix $A \in \mathbb{R}^{n \times n}$ has at least one real-valued eigenvector $\mathbf{v}_1$. Then we show that A preserves the subspace orthogonal to $\mathbf{v}_1$ that is $\mathbf{v}_1' A\mathbf{x} = 0$ iff $\mathbf{v}_1'\mathbf{x} = 0$ and using this fact we iteratively contruct n real-valued orthogonal eigenvectors.

We want to maximise the following (subject to the unit circle)

$$\max_{\mathbf{x}'\mathbf{x}=1} \mathbf{x}' A\mathbf{x},$$

with the lagrangian

$$\mathcal{L}(\mathbf{x}) = \mathbf{x}' A\mathbf{x} - \lambda \left(\mathbf{x}'\mathbf{x} - 1\right),$$

and its first order conditions

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}}|_{\mathbf{x}=\mathbf{x}^*} = (A + A')\mathbf{x}^* - 2\lambda\mathbf{x}^* = 2A\mathbf{x}^* - 2\lambda\mathbf{x}^* = 0$$
$$A\mathbf{x}^* = \lambda\mathbf{x}^*,$$

where the second equality uses symmetry $A = A'$. Thus we have shown that there exists a vector $\mathbf{v}_1 = \mathbf{x}^*$ with the associated eigenvalue λ_1. From here we continue by finding the next eigenvector. We simply solve the same maximisation problem with the added restriction $\mathbf{v}_1'\mathbf{x} = 0$:

$$\max_{\mathbf{x}'\mathbf{x}=1, \mathbf{v}_1'\mathbf{x}=0} \mathbf{x}'A\mathbf{x},$$

with the lagrangian

$$\mathcal{L}(\mathbf{x}) = \mathbf{x}'A\mathbf{x} - \lambda\left(\mathbf{x}'\mathbf{x} - 1\right) - \mu\left(\mathbf{v}_1'\mathbf{x}\right),$$

and the first order condition

$$A\mathbf{x}^* = \lambda\mathbf{x}^* + \mu\mathbf{v}_1 = \lambda\mathbf{x}^*,$$

where the last inequality (i.e. $\mu = 0$) follows from symmetry of A.[2] Therefore we have found another eigenvector which we name $\mathbf{v}_2$ and its associated eigenvalue λ_2. Keep going like this and find all n eigenvectors to obtain

$$A\mathbf{v}_i = \lambda_i\mathbf{v}_i,\ i = 1, \ldots, n,$$

[2]To see this let $\mathbf{v}_1'\mathbf{w} = 0$ then because of symmetry

$$\mathbf{v}_1'A\mathbf{w} = (A\mathbf{v}_1)'\mathbf{w} = (\lambda\mathbf{v}_1)'\mathbf{w} = \lambda\mathbf{v}_1'\mathbf{w} = 0$$

and therefore

$$\underbrace{\mathbf{v}_1'A\mathbf{x}^*}_{=0} = \lambda\underbrace{\mathbf{v}_1'\mathbf{x}^*}_{=0} + \mu\underbrace{\mathbf{v}_1'\mathbf{v}_1}_{=1} \iff \mu = 0.$$

which allows us to write

$$AQ = Q\Lambda,$$

where $Q = \begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_n \end{bmatrix} \in \mathbb{R}^{n \times n}$ and $\Lambda = diag(\lambda_1, \cdots, \lambda_n)$. Now remember that the eigenvectors are constructed iterately where the jth eigenvector is orthogonal to the preceeding $j-1$ eigenvectors.

$$\mathbf{v}_i' \mathbf{v}_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases},$$

which can be collected into a matrix $Q = \begin{bmatrix} \mathbf{v}_1' & \mathbf{v}_2' & \cdots & \mathbf{v}_n' \end{bmatrix}$ such that

$$QQ' = \begin{bmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \\ \vdots \\ \mathbf{v}_n \end{bmatrix} \begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_n \end{bmatrix} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix} = I_n.$$

Therefore we have

$$AQ = Q\Lambda \iff A = Q\Lambda Q'$$

thus completing the proof.

3.2 Overidentified case: $N > rank(X)$

Let the design matrix be defined as

$$X = \begin{bmatrix} \phi(\mathbf{x}_1)' & 1 \\ \vdots & \vdots \\ \phi(\mathbf{x}_n)' & 1 \end{bmatrix} \in \mathbb{R}^{N \times d+1},$$

where $\phi(\mathbf{x}_i) \in \mathbb{R}^d$ is observation $i = 1, \ldots, N$ which is possibly transformed / constructed from the raw data. That is the raw data for person / observation i,

$\mathbf{x}_i$ is used to extract the features $\phi(\mathbf{x}_i) \in \mathbb{R}^d$. For example we could have

$$\phi(\begin{bmatrix} age_i & height_i \end{bmatrix}') = \begin{bmatrix} age_i & age_i^2 & height_i \end{bmatrix}',$$

so that the dimensionality of the feature space is $d = 3$ and larger than the underlying raw datasets 2 dimensions.

Problem: For $N > d + 1$ and no linearly dependent columns in X then $X'X$ has full rank and the following minimisation problem (using the MSE as loss function) has exactely one solution

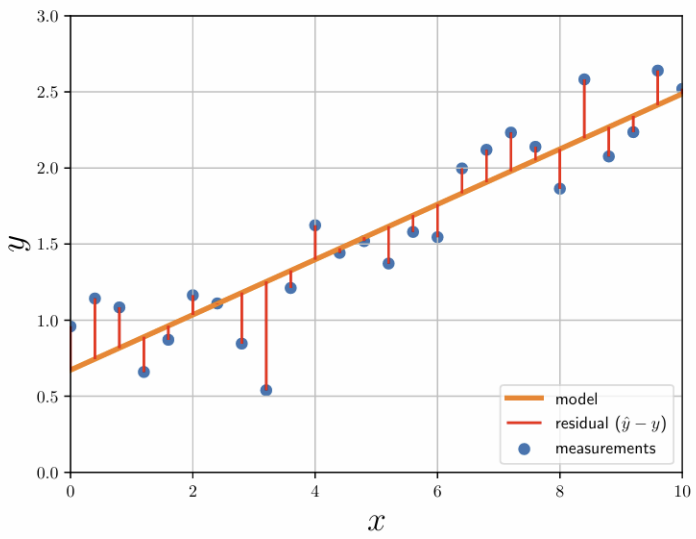
$$\begin{aligned} \hat{\mathbf{w}} =& \arg \min_{\mathbf{w} \in \mathbb{R}^{d+1}} \frac{1}{N} \sum_{i=1}^{N} (y_i - \mathbf{w}' \tilde{\phi}(\mathbf{x}_i))^2 \\ =& \arg \min_{\mathbf{w} \in \mathbb{R}^{d+1}} \frac{1}{N} (\mathbf{y} - X\mathbf{w})' (\mathbf{y} - X\mathbf{w}), \end{aligned}$$

where $\tilde{\phi}(\mathbf{x}_i) = \begin{bmatrix} \phi(\mathbf{x}_i) & 1 \end{bmatrix}' \in \mathbb{R}^{d+1}$ describes the possible transformation of the raw underlying data $\mathbf{x}_i$.

Solution: The first order condition of the problem is

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{w}} &= \frac{2}{N} X'(\mathbf{y} - X\mathbf{w}) = 0 \\ \mathbf{w} &= (X'X)^{-1} X'\mathbf{y}, \end{aligned}$$

which yields the well known OLS estimator. Here the error is visualized:



3.2.1 *OLS is equivalent with Gaussian MLE*

Assume $y_i = f_{\mathbf{w}}(\mathbf{x}_i) + \varepsilon_i$, where $\varepsilon_i \overset{i.i.d.}{\sim} \mathcal{N}(0, \sigma^2)$. Then the likelihood is

$$L(\mathbf{w}) = \prod_{i=1}^{N} p(y_i \mid \mathbf{x}_i; \mathbf{w}) = \prod_{i=1}^{N} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - f_{\mathbf{w}}(\mathbf{x}_i))^2}{2\sigma^2}\right).$$

Taking the negative log-likelihood gives

$$-\log L(\mathbf{w}) = \frac{N}{2}\log(2\pi\sigma^2) + \frac{1}{2\sigma^2}\sum_{i=1}^{N}(y_i - f_{\mathbf{w}}(\mathbf{x}_i))^2.$$

Hence, for fixed σ^2, maximum likelihood estimation is equivalent to minimizing the mean squared error:

$$\hat{\mathbf{w}}_{\text{MLE}} = \arg\max_{\mathbf{w}} L(\mathbf{w}) = \arg\min_{\mathbf{w}} \frac{1}{N}\sum_{i=1}^{N}(y_i - f_{\mathbf{w}}(\mathbf{x}_i))^2.$$

3.3 Underidentified case: $N < rank(X)$

When $N < rank(X)$ the system of equations $\mathbf{w} = (X'X)^{-1}X'\mathbf{y}$ has infinitely many solutions. Therefore we need a way to choose a good $\mathbf{w}$ amongs the many

solutions that satisfy $\mathbf{w}_i' \mathbf{x}_i = y_i,\ i = 1, \ldots, N$.

3.3.1 Minimum-norm solution

Problem: One sensible way to estimate the model is to choose the weighting vector $\mathbf{w}$ with the smallest euclidian norm where the model still fits the training data perfectly:

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w} \in \mathbb{R}^{d+1}} ||\mathbf{w}||_2^2 \text{ s.t. } \forall i : \mathbf{w}_i' \mathbf{x}_i = y_i$$

Solution: We solve the constrained optimization problem using that the linear constraint can be written as $\mathbf{w}_i' \mathbf{x}_i = y_i \iff X\mathbf{w} = \mathbf{y}$.

Setting up the lagrangian

$$\begin{aligned} \mathcal{L}(\mathbf{w}, \boldsymbol{\lambda}) &= \mathbf{w}^\top w - \sum_{n=1}^{N} \lambda_n \left(\mathbf{w}_n' \mathbf{x}_n - y_n \right) \\ &= \mathbf{w}^\top \mathbf{w} - \boldsymbol{\lambda}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}), \end{aligned}$$

First-order condition with respect to $\mathbf{w}$ gives

$$\nabla_{\mathbf{w}} \mathcal{L} = 2\mathbf{w} - X^\top \boldsymbol{\lambda} = 0 \quad \Longrightarrow \quad \mathbf{w} = \frac{1}{2} X^\top \boldsymbol{\lambda}.$$

Substituting into the constraint yields

$$X\mathbf{w} = \mathbf{y} \quad \Longrightarrow \quad \frac{1}{2} X X^\top \boldsymbol{\lambda} = \mathbf{y} \quad \Longrightarrow \quad \boldsymbol{\lambda} = 2(XX^\top)^{-1} \mathbf{y}.$$

Hence, the minimum-norm solution is

$$\hat{\mathbf{w}} = \underbrace{X^\top (XX^\top)^{-1}}_{= X^\dagger} \mathbf{y}.$$

3.3.2 *Singular Value Decomposition and the Pseudoinverse*

Any matrix $X \in \mathbb{R}^{N \times d}$ admits a singular value decomposition (SVD)

$$X = U\Gamma V^\top,$$

where, for $r = \min\{N, d\}$, $U \in \mathbb{R}^{N \times r}$ and $V \in \mathbb{R}^{d \times r}$ are semi-orthogonal, and $\Gamma = \operatorname{diag}(\sigma_1, \ldots, \sigma_r)$ contains the singular values $\sigma_1 \geq \cdots \geq \sigma_r \geq 0$.

Ordinary Least Squares ($N \geq d$). Assuming full column rank, the OLS estimator is

$$\begin{aligned}\hat{\mathbf{w}}_{\text{OLS}} &= (X^\top X)^{-1} X^\top \mathbf{y} \\ &= (V\Gamma^2 V^\top)^{-1} V\Gamma U^\top \mathbf{y} \\ &= V\Gamma^{-1} U^\top \mathbf{y}.\end{aligned}$$

Minimum-Norm Interpolation ($N \leq d$). Assuming full row rank, the minimum-norm solution satisfying $X\mathbf{w} = \mathbf{y}$ is

$$\begin{aligned}\hat{\mathbf{w}}_{\text{MN}} &= X^\top (XX^\top)^{-1} \mathbf{y} \\ &= V\Gamma U^\top (U\Gamma^2 U^\top)^{-1} \mathbf{y} \\ &= V\Gamma^{-1} U^\top \mathbf{y}.\end{aligned}$$

Thus, both estimators can be expressed using the **Moore-Penrose pseudoinverse**:

$$\hat{\mathbf{w}} = X^\dagger \mathbf{y}, \qquad X^\dagger = V\Gamma^\dagger U^\top,$$

where $\Gamma^\dagger$ is diagonal with entries $1/\sigma_i$ if $\sigma_i > 0$ and 0 otherwise. This also holds for rank-deficient matrices, in which case the pseudoinverse selects the minimum-norm least-squares solution.

Why use SVD? Computing $(X^\top X)^{-1}$ directly can be numerically unstable because forming $X^\top X$ squares the condition number:

$$\kappa(X^\top X) = \kappa(X)^2.$$

SVD avoids this squaring and handles rank deficiency by inverting only nonzero singular values. However, very small singular values can still amplify numerical errors, motivating regularization or truncation.

3.3.3 Ridge regression (Two-norm regularization)

Problem: Add regularisation term $\lambda||\mathbf{w}||_2^2$ with $\lambda > 0$ and the problem of ridge regression becomes

$$\begin{aligned}\hat{\mathbf{w}} &= \arg\min_{\mathbf{w}\in\mathbb{R}^{d+1}} \frac{1}{N}\sum_{i=1}^{N}(y_i - \mathbf{w}'\tilde{\boldsymbol{\phi}}(\mathbf{x}_i))^2 + \lambda||\mathbf{w}||_2^2 \\ &= \arg\min_{\mathbf{w}\in\mathbb{R}^{d+1}} \frac{1}{N}(\mathbf{y} - X\mathbf{w})'(\mathbf{y} - X\mathbf{w}) + \lambda\mathbf{w}'\mathbf{w},\end{aligned}$$

Solution: The first order condition (of the lagrangian) are

$$\begin{aligned}\frac{\partial L}{\partial \mathbf{w}}|_{\mathbf{w}=\hat{\mathbf{w}}} &= \frac{2}{N}\left(X'(\mathbf{y} - X\hat{\mathbf{w}}) + \lambda\hat{\mathbf{w}}\right) = 0 \\ \hat{\mathbf{w}} &= (X'X + \lambda I_{d+1})X'\mathbf{y}.\end{aligned}$$

Note that for $\lambda \to 0$ the solution approaches that of OLS and for $\lambda \to \infty$ the solution goes to 0 $\hat{\mathbf{w}} \to \mathbf{0}$. Intuitively this is because weight-elements other than 0 are penalised a lot.

Choosing regularization paramter: λ can be chosen by cross validation (choosing the λ^* minimzing CV risk).

3.3.4 Equivalence theorem

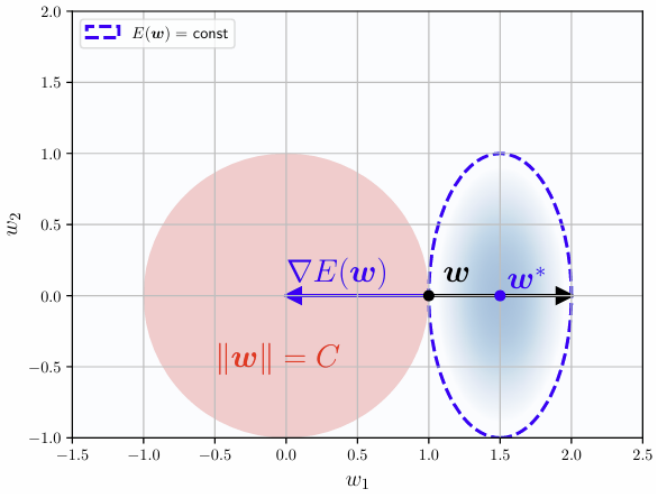
Ridge regression can be seen as solving

$$\arg\min_{\mathbf{w}\in\mathbb{R}^{d+1}} \frac{1}{N}\sum_{i=1}^{N}(y_i - \tilde{\boldsymbol{\phi}}(\mathbf{x}_i))^2 \text{ s.t. } ||\mathbf{w}||_2 \leq C,$$

becuase this is equivalent to the ridge regression problem:

$$\arg \min_{\mathbf{w} \in \mathbb{R}^{d+1}} \frac{1}{N} (\mathbf{y} - X\mathbf{w})' (\mathbf{y} - X\mathbf{w}) + \lambda \mathbf{w}'\mathbf{w},$$

for some $\lambda = f(C)$ depending on C. This is visualized here



This intepretation is useful because of what the restricted hypothesis space (where $||\mathbf{w}||_2 \leq C$) tries to implement for the affine linear hypothesis $\tilde{\boldsymbol{\phi}}(\mathbf{x}_i)$). Formally let $\tilde{\phi} : \mathbb{R}^{d_0} \to \mathbb{R}^{d+1}$ (where d_0 is the dimensions of the raw underlying data which the featurespace $\mathbb{R}^{d+1}$ is created from). be continuous and Lipschits with constant α meaning:

$$||\tilde{\boldsymbol{\phi}}(\mathbf{x}_1) - \tilde{\boldsymbol{\phi}}(\mathbf{x}_2)||_2 \leq \alpha ||\mathbf{x}_1 - \mathbf{x}_2||_2,$$

for $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^{d_0}$. Then the functions $h : \mathbb{R}^{d_0} \to \mathbb{R}^1$ in

$$\mathcal{H}_C = \left\{ h(\mathbf{x}) = \mathbf{w}'\tilde{\boldsymbol{\phi}}(\mathbf{x}) \mid \mathbf{w} \in \mathbb{R}^{d+1} \text{ and } ||\mathbf{w}||_2 \leq C \right\},$$

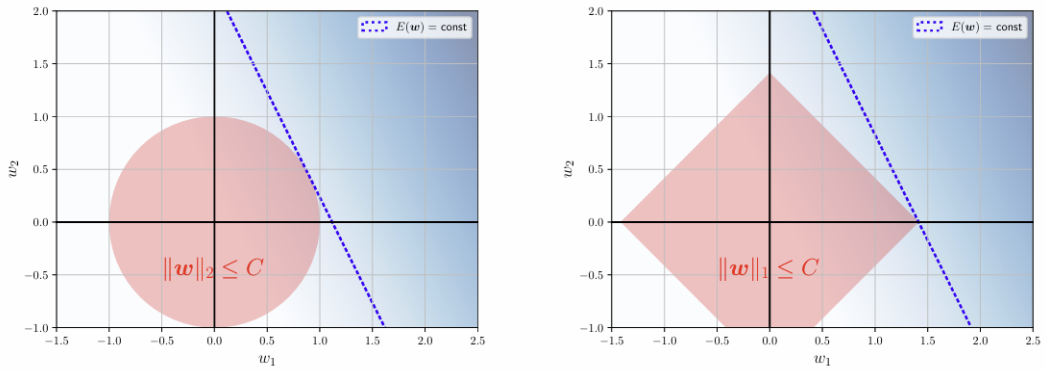
will be Lipschitz with constant $C \cdot \alpha$. In other words C limits how fast the prediction rule / hypothesis h can change.

3.3.5 *Lasso regression (One-norm regularization)*

Much the same as ridge regression but where we add the l1-norm regularization (to reduce the number of active dimensions - i.e. increase number of weights equal to 0 in $\mathbf{w}$)

$$\begin{aligned}\hat{\mathbf{w}} &= \arg\min_{\mathbf{w}\in\mathbb{R}^{d+1}} \frac{1}{N}\sum_{i=1}^{N}(y_i - \mathbf{w}'\tilde{\boldsymbol{\phi}}(\mathbf{x}_i))^2 + \lambda||\mathbf{w}||_1 \\ &= \arg\min_{\mathbf{w}\in\mathbb{R}^{d+1}} \frac{1}{N}(\mathbf{y} - X\mathbf{w})'(\mathbf{y} - X\mathbf{w}) + \lambda\sum_{j=1}^{d}|w_j|\end{aligned}$$

The sparcity effect is visualized here



3.4 Comment on double descent

Sometimes when letting the number of parameters grow $d >> N$ - while including a regularization term like $\lambda||\mathbf{w}||_2^2$, $\lambda > 0$ then at some point testing error can start diminishing rather than keep increasing. This can even result in lower empirical test loss than models with $d < N$.

The name comes from the stylized fact that test loss decreases when adding model dimensions d and d is low. At some point the model overfits and testing empirical risk increases. And then the magic can enter for $d >> N$ where the test loss again decreases!

4 Linear Classification

The task of linear classification is to predict the correct class $\mathcal{Y} \in \{\mathcal{C}_1, \ldots, \mathcal{C}_m\}$, $2 \leq m$ from the input $\mathbf{x}_i \in \mathcal{X}$. One approach is to learn the discriminant functions $\delta_k : \mathcal{X} \to \mathbb{R}$, $k = 1, \ldots, m$

$$\hat{y}_i = h(\mathbf{x}_i) = \arg \max_{k \in \{1,\ldots,m\}} \delta_k(\mathbf{x}_i)$$

We specify affine linear discriminant functions

$$\delta(\mathbf{x}_i) = \mathbf{w}' \boldsymbol{\phi}(\tilde{\mathbf{x}}_i),$$

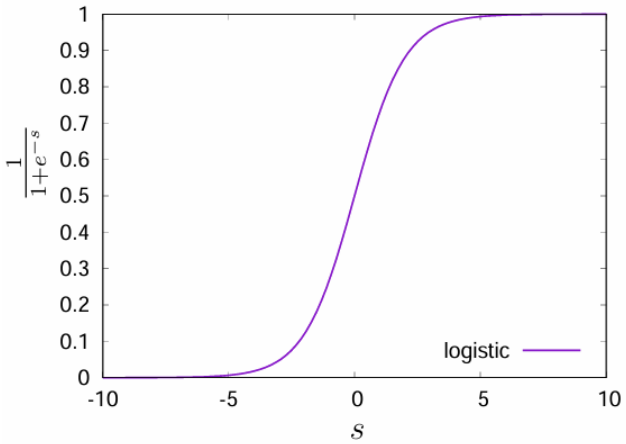
where $\mathbf{x}_i = \boldsymbol{\phi}(\tilde{\mathbf{x}}_i)$ is the features always augmentet to hold a 1 (so that the model is affine linear - just like in linear regression) and can also hold many other features created from the raw data $\tilde{\mathbf{x}}_i$.

4.1 Logistic regression

4.1.1 Binary case

Let $\mathcal{Y} \in \{1, -1\}$ and let $\theta : \mathbb{R} \to [0, 1]$ be the sigmoid function defined by

$$\theta(s) = \frac{1}{1 + \exp\{s\}}.$$



Deriving the binary sigmoid: The sigmoid encodes $\mathbf{w}'\mathbf{x}$ as the log ratio of probabilities. i.e. on log scale how relatively probable $Y = 1$ is compared to $Y = -1$.

$$\begin{aligned}
\mathbf{w}'\mathbf{x} &= \ln \frac{\mathbb{P}(Y = 1 \mid X = \mathbf{x})}{\mathbb{P}(Y = -1 \mid X = \mathbf{x})} \\
\exp\{\mathbf{w}'\mathbf{x}\} &= \frac{\mathbb{P}(Y = 1 \mid X = \mathbf{x})}{1 - \mathbb{P}(Y = 1 \mid X = \mathbf{x})} \\
(1 - \mathbb{P}(Y = 1 \mid X = \mathbf{x}))\exp\{\mathbf{w}'\mathbf{x}\} &= \mathbb{P}(Y = 1 \mid X = \mathbf{x}) \\
\mathbb{P}(Y = 1 \mid X = \mathbf{x}) &= \frac{\exp\{\mathbf{w}'\mathbf{x}\}}{1 + \exp\{\mathbf{w}'\mathbf{x}\}} \\
&= \frac{1}{1 + \exp\{-\mathbf{w}'\mathbf{x}\}} \equiv \theta(\mathbf{w}'\mathbf{x}).
\end{aligned}$$

Properties of sigmoid in the binary case: In the binary case we have

$$\theta(y\mathbf{w}'\mathbf{x}) = \begin{cases} \theta(\mathbf{w}'\mathbf{x}) & \text{if } y = 1 \\ 1 - \theta(\mathbf{w}'\mathbf{x}) & \text{if } y = -1 \end{cases}.$$

This is easy to show:

$$\begin{aligned}
1 - \theta(\mathbf{w}'\mathbf{x}) &= \frac{1 + \exp\{-\mathbf{w}'\mathbf{x}\}}{1 + \exp\{-\mathbf{w}'\mathbf{x}\}} - \frac{1}{1 + \exp\{-\mathbf{w}'\mathbf{x}\}} \\
&= \frac{\exp\{-\mathbf{w}'\mathbf{x}\}}{1 + \exp\{-\mathbf{w}'\mathbf{x}\}} \\
&= \frac{1}{1 + \exp\{\mathbf{w}'\mathbf{x}\}} \\
&= \theta(-\mathbf{w}'\mathbf{x}) \\
&= \theta(y\mathbf{w}'\mathbf{x}),\ y = -1
\end{aligned}$$

Log likelihood optimization: Therefore the negative log likelihood function

becomes

$$\begin{aligned}
-\ln L(\mathbf{w}) &= -\ln \prod_{n=1}^{N} \mathbb{P}\left(Y = y_n \mid X = \mathbf{x}_n; \mathbf{w}\right) \\
&= -\sum_{n=1}^{N} \ln \mathbb{P}\left(Y = y_n \mid X = \mathbf{x}_n; \mathbf{w}\right) \\
&= -\sum_{n=1}^{N} \ln \frac{1}{1+\exp\{-y_n \mathbf{w}'\mathbf{x}_n\}} \\
&= \sum_{n=1}^{N} \ln[1+\exp\{-y_n \mathbf{w}'\mathbf{x}_n\}],
\end{aligned}$$

which when divided by N is the objective function of the unconstrained minimisation problem

$$\hat{\mathbf{w}}_{MLE} = \arg\min_{\mathbf{w}\in\mathbb{R}^{d+1}} \underbrace{-\frac{1}{N}\sum_{n=1}^{N} \ln[1+\exp\{-y_n \mathbf{w}'\mathbf{x}_n\}]}_{Q(\mathbf{w})},$$

with first order condition

$$\begin{aligned}
\nabla_{\mathbf{w}} Q(\mathbf{w}) &= \frac{1}{N}\sum_{n=1}^{N} \frac{1}{1+\exp\{-y_n \mathbf{w}'\mathbf{x}_n\}} \left(-\exp\{-y_n \mathbf{w}'\mathbf{x}_n\}\right)(-y_n \mathbf{x}_n) \\
&= \frac{1}{N}\sum_{n=1}^{N} \frac{y_n \mathbf{x}_n}{1+\exp\{y_n \mathbf{w}'\mathbf{x}_n\}} = 0,
\end{aligned}$$

which does not have any closed form solution. However, because the problem is convex this can be solves globally using *gradient descent*. The only caveat is the case where data is perfectly linearly seperable. Then $Q(\mathbf{w}) \to 0$ as $\mathbf{w} \to \infty$ and the problem has no solution. Instead one can solve the L2-norm regularized problem (just like in linear regression) on the form

$$\hat{\mathbf{w}}_{MLE,L2} = \arg\min_{\mathbf{w}\in\mathbb{R}^{d+1}} Q(\mathbf{w}) + \lambda ||\mathbf{w}||_2^2,\ \lambda > 0,$$

which is a stricly convex problem.

4.1.2 *Multiple classification*

Defining the probability distribution: The only difference is how the probability distribution is defined. The MLE machinery + gradient descent always stays the same. For each category

$$\mathcal{Y} \in \{\mathcal{C}_1, \ldots, \mathcal{C}_m\},\ 2 \leq m$$

we define

$$\mathbb{P}\left(Y = y \mid X = \mathbf{x}; \mathbf{w}_1, \ldots, \mathbf{w}_m\right) \equiv \frac{\exp\{\delta_k(\mathbf{x})\}}{\sum_{j=1}^{m} \exp\{\delta_j(\mathbf{x})\}} = \frac{\exp\{\mathbf{w}_k'\mathbf{x}\}}{\sum_{j=1}^{n} \exp\{\mathbf{w}_j'\mathbf{x}\}}.$$

Log likelihood: This yields the nogative log likelihood function

$$\begin{aligned}
-\ln L(\mathbf{w}_1, \ldots, \mathbf{w}_m) &= -\ln \prod_{n=1}^{N} \mathbb{P}\left(Y = y_n \mid X = \mathbf{x}_n; \mathbf{w}_1, \ldots, \mathbf{w}_m\right) \\
&= -\sum_{n=1}^{N} \ln \mathbb{P}\left(Y = y_n \mid X = \mathbf{x}_n; \mathbf{w}_1, \ldots, \mathbf{w}_m\right) \\
&= -\sum_{n=1}^{N} \ln \frac{\exp\{\mathbf{w}_k'\mathbf{x}_n\}}{\sum_{j=1}^{n} \exp\{\mathbf{w}_j'\mathbf{x}_n\}}.
\end{aligned}$$

Identification: Since adding any vector $\mathbf{a} \in \mathbb{R}^{d+1}$ to $\mathbf{w}_k$, $i = 1, \ldots, m$ does not change the (negative log) likelihood:

$$\frac{\exp\{(\mathbf{w}_k + \mathbf{a})'\mathbf{x}_n\}}{\sum_{j=1}^{n} \exp\{(\mathbf{w}_j + \mathbf{a})'\mathbf{x}_n\}} = \frac{\exp\{\mathbf{a}\}\exp\{\mathbf{w}_k'\mathbf{x}_n\}}{\exp\{\mathbf{a}\}\sum_{j=1}^{n} \exp\{\mathbf{w}_j'\mathbf{x}_n\}} = \frac{\exp\{\mathbf{w}_k'\mathbf{x}_n\}}{\sum_{j=1}^{n} \exp\{\mathbf{w}_j'\mathbf{x}_n\}},$$

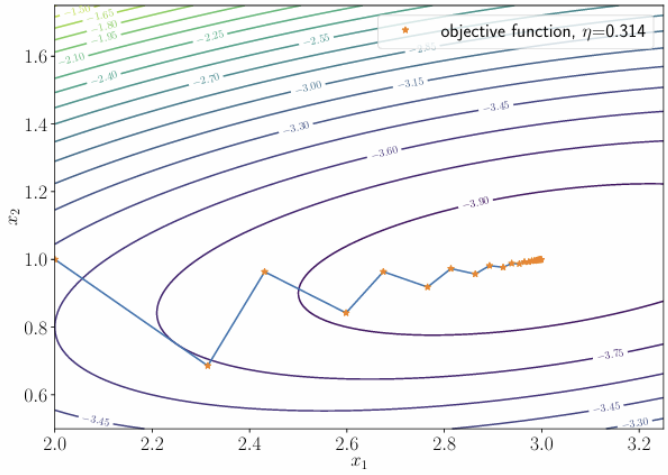
then we either need to add a regularization term or normalise one of the class-weights. For the latter solution a common choice is $\mathbf{w}_m = \mathbf{0} \in \mathbb{R}^{d+1}$. The remaining model-weights $\hat{\mathbf{w}}_1, \ldots, \hat{\mathbf{w}}_{m-1}$ are solved for using gradient descent.

4.1.3 *Optimization: Gradient descent methods*

Gradient descent: In short gradient descent iteratively updates the weights using a first order taylor expansion around the current point $\mathbf{w}^{(t)}$, where (t) indicates the iteration number. The goal is to minimize $Q(\mathbf{w})$.

$$\begin{aligned}\mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} + \boldsymbol{\Delta}_{\mathbf{w}}{}^{(t)} \\ &= \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} Q|_{\mathbf{w}^{(t)}},\end{aligned}$$

with learning rate $\eta > 0$ determining how large steps are in proportion to the gradient.



Often we add a *momentum term* (in an attempt to improve performance) which intuitively works like a heavy ball. When a heavy ball has gained momentum from accelerating down a steep hill it will roll over local minimums and further down towards the global minimum (hopefully). Formally we have

$$\begin{aligned}\mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} - \eta \nabla E|_{\mathbf{w}^{(t)}} + \mu \boldsymbol{\Delta}_{\mathbf{w}}{}^{(t-1)} \\ &= \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} Q|_{\mathbf{w}^{(t)}} + \mu \left(\mathbf{w}^{(t)} - \mathbf{w}^{(t-1)} \right),\end{aligned}$$

with momentum parameter $\mu > 0$.

Stochastic gradient descent: Esentially the same algorithm but where the gradi-

ent is calculated using one $(\mathbf{x}_n, y_n) \in S$, where S is the training data.

Minibatch gradient descent: The same but where the gradient is calculated on a random subset $S' \subset S$.

5 Random Forest

5.1 Binary decision trees (definitions)

CART trees are introduced and used in MLA and are popular binary classification / regression trees. Each *inner node* in a CART tree evaluates one dimension $d \in \{1, \ldots, D\}$ of the dataset using a threshold value θ and splits the data into

$$L_{d,\theta} = \{(\mathbf{x}, y) \in S \mid x_d < \theta\} \text{ and } R_{d,\theta} = \{(\mathbf{x}, y) \in S \mid x_d \geq \theta\},$$

where x_d is element d of $\mathbf{x}$. The subset $L_{d,\theta}$ is then sent to the left daughter node and $R_{d,\theta}$ to the right.

The splits are determined by maximising the information gain or equivalently minimising the impurity measure Q at the dataset at that note S. Formally we would like to choose the dimension d as well as the threshold value θ solving

$$\begin{aligned} &\arg \max_{d,\theta} Q(S) - \frac{|L_{d,\theta}|}{|S|} Q(L_{d,\theta}) - \frac{|R_{d,\theta}|}{|S|} Q(R_{d,\theta}) \\ = &\arg \min_{d,\theta} G_{d,\theta}(S) = |L_{d,\theta}| Q(L_{d,\theta}) + |R_{d,\theta}| Q(R_{d,\theta}). \end{aligned}$$

This is however impractical (and maybe even impossible) to solve on a computer and we therefore use some heuristic.

Leaf nodes are indexed by $\tau = 1, \ldots, |T|$ and they define *regions*. If $\mathbf{x}_i$ reaches leaf node τ we have $\mathbf{x} \in \mathcal{R}_\tau$.

5.1.1 Regression trees

Trees can not only classify observations $\mathbf{x}_i$ into categories but also estimate its value (if the label is continuous)

$$\hat{y}_i = T(\mathbf{x}_i) = \sum_{\tau=1}^{|T|} c_\tau \mathbb{I}(\mathbf{x}_i \in \mathcal{R}_\tau)$$

With squared loss $(\hat{y} - y)^2$ the constants minimising empirical risk are the respective empirical means of each leafnode τ:

$$c_\tau = \frac{1}{N_\tau} \sum_{\mathbf{x}_n \in \mathcal{R}_\tau} y_n,$$

with $(\mathbf{x}_n, y) \in S$ and $N_\tau = |\{(\mathbf{x}_n, y) \in S \mid \mathbf{x}_n \in \mathcal{R}_\tau\}|$.

5.1.2 Classification trees

For K classes, let $p_{\tau k}$ be the fraction of training observations in leaf τ with label k. The tree predicts the most frequent class in the leaf,

$$T(\mathbf{x}) = \underset{k \in \{1,\ldots,K\}}{\arg\max}\, p_{\tau k}, \qquad \mathbf{x} \in \mathcal{R}_\tau.$$

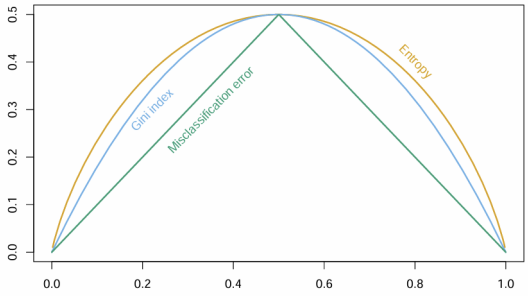
The fractions $p_{\tau k}$ also give estimated class probabilities. Common impurity measures are

$$Q_{\text{error}} = 1 - \max_k p_{\tau k},$$

$$Q_{\text{entropy}} = -\sum_{k=1}^{K} p_{\tau k} \log p_{\tau k},$$

$$Q_{\text{Gini}} = \sum_{k=1}^{K} p_{\tau k}(1 - p_{\tau k}),$$

where $0 \log 0 = 0$. All three are zero when the node is pure.



5.1.3 *Growing and pruning*

For each dimension d, it is enough to test thresholds halfway between consecutive distinct values of x_d. We split recursively until the node is pure, too small or has no valid split. For regression, Q is the mean squared deviation from the node mean.

For a single CART tree, we can prune by choosing a subtree T that minimises

$$C_\alpha(T) = \sum_{\tau=1}^{|T|} N_\tau Q_\tau(T) + \alpha|T|, \qquad \alpha \geq 0,$$

where $Q_\tau(T)$ is the impurity in leaf τ. Larger α penalises more leaves. We choose α using held-out data.

5.2 Ensembles and bagging

Single trees have high variance, so small changes in the training data can give very different trees. Bagging trains trees on separate bootstrap samples and averages their predictions (or votes).

If B regression trees have mean-zero errors ϵ_b with variance v and pairwise covariance c, then

$$\mathbb{E}\left[\left(\frac{1}{B}\sum_{b=1}^{B}\epsilon_b\right)^2\right] = \frac{v}{B} + \frac{B-1}{B}c.$$

Thus averaging helps more when the trees make less correlated errors. For binary classification, if errors are independent and each tree errs with probability at most $p_{\max} < 1/2$, Hoeffding bounds the majority vote error by

$\exp[-2B(1/2 - p_{\max})^2]$.

5.3 Growing and evaluating a random forest

For each $b = 1, \ldots, B$ we draw a bootstrap sample S_b by sampling N observations with replacement from S. We grow an unpruned tree T_b on S_b. At each node we draw a fresh subset of m distinct dimensions and find the best split among these. This makes the trees less correlated. With $m = D$ we recover bagging of trees.

For regression we average the predictions. For classification we take the most votes, or average the leaf probabilities and take the most probable class. Formally, these three rules are

$$
\begin{aligned}
f_{\mathrm{RF}}^{\mathrm{reg}}(\mathbf{x}) &= \frac{1}{B} \sum_{b=1}^{B} T_b(\mathbf{x}), \\
f_{\mathrm{RF}}^{\mathrm{vote}}(\mathbf{x}) &= \arg\max_{k} \sum_{b=1}^{B} \mathbb{I}\big(T_b(\mathbf{x}) = k\big), \\
f_{\mathrm{RF}}^{\mathrm{prob}}(\mathbf{x}) &= \arg\max_{k} \frac{1}{B} \sum_{b=1}^{B} p_k^{(b)}(\mathbf{x}),
\end{aligned}
$$

where $p_k^{(b)}(\mathbf{x})$ is the fraction of class k in the leaf of T_b reached by $\mathbf{x}$.

Typical starting values are $m = \lfloor \sqrt{D} \rfloor$ for classification, $m = \max\{1, \lfloor D/3 \rfloor\}$ for regression and $B = 100$ trees.

5.4 Out-of-bag (OOB) error

An observation i is out of bag for tree b if its index was never drawn for S_b. This happens with probability

$$
\left(1 - \frac{1}{N}\right)^N \approx e^{-1} \approx 0.37.
$$

Let $\mathcal{B}_i$ be the set of trees for which i is out of bag. We predict $\mathbf{x}_i$ using the usual forest rule restricted to these trees. With $\mathcal{I} = \{i : \mathcal{B}_i \neq \varnothing\}$, the OOB estimate of

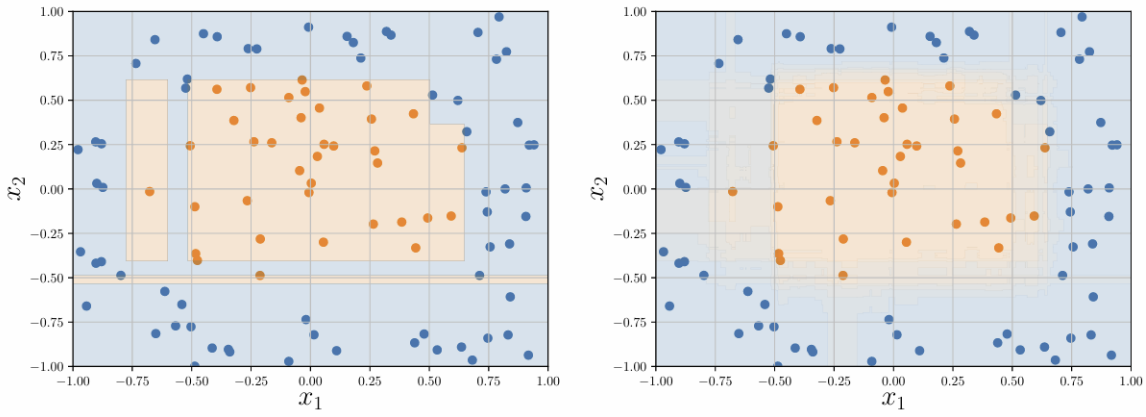
generalisation error is

$$\widehat{R}_{\mathrm{OOB}} = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \ell\big(y_i, f_{\mathrm{RF}}(\{T_b : b \in \mathcal{B}_i\}, \mathbf{x}_i)\big).$$

We can choose m using OOB error and increase B until it stabilises.

5.5 Discussion: CART vs Random Forest

Single trees are easy to interpret but are unstable and have piecewise constant predictions. Random forests usually perform well on tabular data with little tuning, and the trees can be trained in parallel. The forest is harder to interpret and still uses axis-aligned splits. Class probabilities may be poorly calibrated.

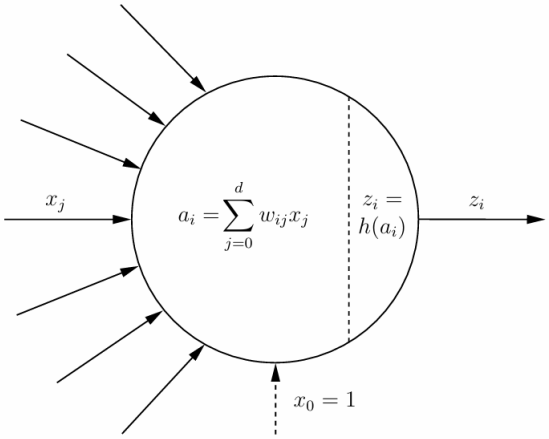


tree (left) vs. random forest (right)

6 Neural Networks

6.1 The Neuron

This chapter explains in detail the following picture which illustrates a single neuron with implicit bias. So if one understands the picture the chapter can be skipped.



6.1.1 *Pre-Activation*

The preactivation of neuron i is given by

$$\begin{aligned} a_i(\mathbf{x}) &= \sum_{j=1}^{d} w_{ij} x_j + w_{i0} \\ &= \mathbf{w}_i' \mathbf{x}_{+1}, \end{aligned}$$

where $\mathbf{x}_{+1} \in \mathbb{R}^{d+1}$ denotes the argmentet vector

$$\mathbf{x}_{+1} = \begin{bmatrix} 1 & x_1 & \cdots & x_d \end{bmatrix}'.$$

6.1.2 *Activation and output*

Applying a (non-linear) activation function $h_i : \mathbb{R} \to \mathbb{R}$ yields the output of neuron i

$$z_i(\mathbf{x}) = h_i(a_i(\mathbf{x})) = h_i \left(\sum_{j=1}^{d} w_{ij} x_j + w_{i0} \right).$$

That is each neuron z_i has its own weights (and bias) $\mathbf{w} \in \mathbb{R}^{d+1}$ and activation function h_i. The latter is usually the same for all neurons except the output-neurons (in the top layer).

6.1.3 *Activation functions*

Linear activation function: For the network structure to have any reason to exist the activation functions h_i must be non-linear. If not the neural network $f : \mathbb{R}^d \to \mathbb{R}^K$ will simply be a linear function of the inputs aswell.

To see this note that each neuron in the inputlayer would output the following functional form

$$\begin{aligned} h\left(\sum_{j=1}^{d} w_{ij}x_j + w_{i0}\right) &= \sum_{j=1}^{d} \underbrace{aw_{ij}}_{\tilde{w}_{ij}} x_j + \underbrace{aw_{i0} + b}_{\tilde{w}_{i0}} \\ &= \sum_{j=1}^{d} \tilde{w}_{ij}x_j + \tilde{w}_{i0}, \end{aligned}$$

and since each pre-activation is simply a affine linear function of the previous

affine linear functions all later neurons will output something of the form:

$$
\begin{aligned}
h\left(a\left(\begin{bmatrix} z_1 & z_2 & \cdots & z_K \end{bmatrix}\right)\right) &= h\left(\sum_{k=1}^{K} w_k z_k + w_0\right) \\
&= \sum_{k=1}^{K} \hat{w}_k z_k + \hat{w}_0 \\
&= \sum_{k=1}^{K} \hat{w}_k \left(\sum_{j=1}^{d} \tilde{w}_{kj} x_j + \tilde{w}_{k0}\right) + \hat{w}_0 \\
&= \sum_{k=1}^{K} \hat{w}_k \sum_{j=1}^{d} \tilde{w}_{kj} x_j + \underbrace{\sum_{k=1}^{K} \hat{w}_k \tilde{w}_{k0} + \hat{w}_0}_{\bar{w}_0} \\
&= \sum_{k=1}^{K} \sum_{j=1}^{d} \hat{w}_k \tilde{w}_{kj} x_j + \bar{w}_0 \\
&= \sum_{j=1}^{d} \sum_{k=1}^{K} \hat{w}_k \tilde{w}_{kj} x_j + \bar{w}_0 \\
&= \sum_{j=1}^{d} x_j \underbrace{\sum_{k=1}^{K} \left(\hat{w}_k \tilde{w}_{kj}\right)}_{\bar{w}_j} + \bar{w}_0 \\
&= \sum_{j=1}^{d} \bar{w}_j x_j + \bar{w}_0.
\end{aligned}
$$

This then also goes for all other layers recieving these outputs (by the exact same argument as above). So in summary: if h_i is linear for all i then there is no purpose to the neural network structure and the simple affine linear model of linear regression fits the model just as well.

Well known activation functions (greatest hits). Here i list some of the most common activation functions. Some are more common for outputlayers for example logistic activation and are rarely used in hidden layers.

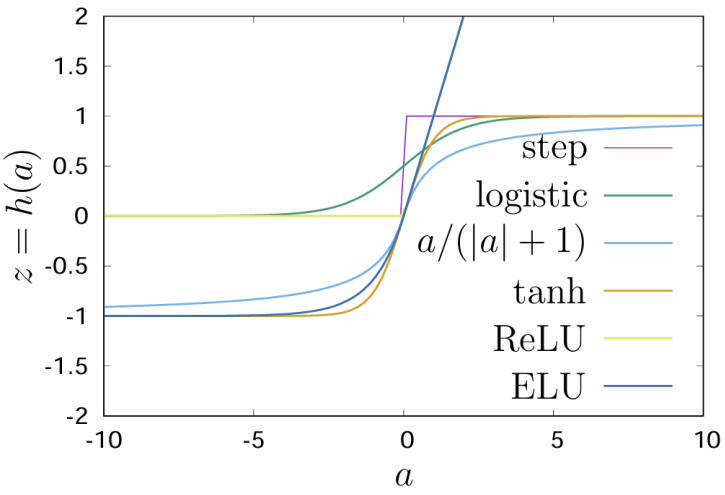
- Step / Threshold

  $$h(a) = \mathbb{I}(a > 0),$$

  with range $\mathcal{R} \in \{0, 1\}$

- Hyperbolic tangent
$$h(a) = tanh(a),$$
with range $\mathcal{R} \in (-1, 1)$ and with a visual form much like *alternative sigmoid.*
- Rectified linear unit (ReLU)
$$h(a) = \max(0, a),$$
with range $\mathcal{R}[0, \infty)$. Note that there the derivative of the loss function wrt. pre-activation weights is uninformative in the regrion $a < 0$. By that i mean $\partial h_i / \partial w_{ij} = 0$ if $a_i(\mathbf{x}, \mathbf{w}) < 0$ and so it does not help us adjust weights to decrease loss in these negative regions.
- Fermi / logistic
$$h(a) = \frac{1}{1 + \exp\{-a\}},$$
with range $\mathcal{R} \in (0, 1)$.
- Alternative sigmoid
$$h(a) = \frac{a}{1 + |a|},$$
with range $\mathcal{R}$
- Exponential LU (ELU, $\alpha > 0$)
$$h(a) = \begin{cases} a & a \geq 0 \\ \alpha(\exp\{a\} - 1) & a < 0 \end{cases}$$
with range $\mathcal{R} \in (-\alpha, \infty)$.

Here the functions are plottet where for ELU $\alpha = 1$ is chosen.

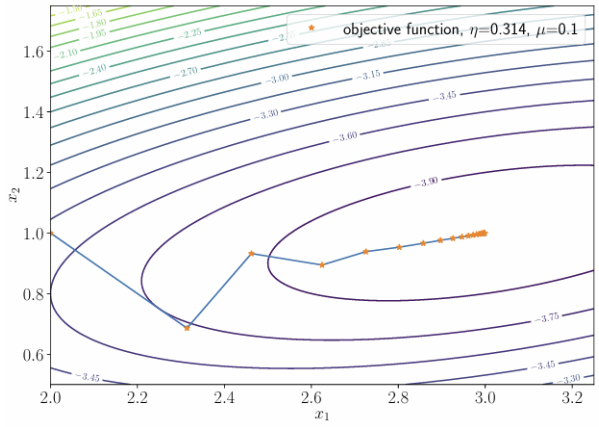


6.2 Gradient descent

To optimize a neural network with error function $E = \sum_{n=1}^{N} E_n$ (as with loglikelihood functions) we introduce gradient descent. The bare form of this algorithm is simply

$$\begin{aligned}\mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} + \boldsymbol{\Delta}_{\mathbf{w}}{}^{(t)} \\ &= \mathbf{w}^{(t)} - \eta \nabla E|_{\mathbf{w}^{(t)}},\end{aligned}$$

with learning rate $\eta > 0$ determining how large steps are in proportion to the gradient.



Often we add a *momentum term* (in an attempt to improve performance) which

intuitively works like a heavy ball. When a heavy ball has gained momentum from accelerating down a steep hill it will roll over local minimums and further down towards the global minimum (hopefully). Formally we have

$$\begin{aligned}\mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} - \eta \nabla E|_{\mathbf{w}^{(t)}} + \mu \boldsymbol{\Delta}_{\mathbf{w}}{}^{(t-1)} \\ &= \mathbf{w}^{(t)} - \eta \nabla E|_{\mathbf{w}^{(t)}} + \mu \left(\mathbf{w}^{(t)} - \mathbf{w}^{(t-1)} \right),\end{aligned}$$

with momentum parameter $\mu > 0$.

The question now really is how to calculate the elements of $\nabla E|_{\mathbf{w}^{(t)}}$ given by

$$\frac{\partial E}{\partial w_{ij}} = \sum_{n=1}^{N} \frac{\partial E_n}{\partial w_{ij}},$$

for $(j, i) \in \mathcal{A}$. Recall that $\mathcal{A}$ contains all the edges connection the output of neuron j to the input of neuron i which is scaled by w_{ij}. This is answered in the next section.

6.3 Backpropagation

The goal is to calculate $\frac{\partial E_n}{\partial w_{ij}}$ for every $(j, i) \in \mathcal{A}$.[3] Applying the chain rule to every element of the error function $E = \sum_{n=1}^{N} E_n$ yields the following. Note that E_n is a function of z_i for all i in the network. and that any z_i is a function of z_j where $(j, i) \in \mathcal{A}$. Therefore the chain rule yields

$$\begin{aligned}\frac{\partial E_n}{\partial w_{ij}} &= \frac{\partial E_n}{\partial z_i} \frac{\partial z_i}{\partial a_i} \frac{\partial a_i}{\partial w_{ij}} \\ &= \frac{\partial E_n}{\partial z_i} h_i'(a_i) \left[\mathbf{z}_n \right]_j\end{aligned}$$

[3] Why not for $(j, i) \neq \mathcal{A}$? These weights simply do not exist. The model only has weights symbolizing the strength of the connect from neuron j to neuron i. There is no connection from i back to j so this w_{ji} does not exist.

where the last step uses that

$$z_i(\mathbf{z}) \equiv h_i(a_i(\mathbf{z})) \implies \frac{\partial z_i}{\partial a_i} = h_i'(a_i),$$

where $\mathbf{z} \in \mathbb{R}^M$ contains all the inputs from the previous layer to neuron i. For the last factor we used that

$$a_i(\mathbf{z}_n) = w_{i0} + \sum_{m=1}^{M} w_{im} \left[\mathbf{z}_n\right]_m \implies \frac{\partial a_i}{\partial w_{ij}} = \left[\mathbf{z}_n\right]_j.$$

Therefore the missing factor to be calculated is $\frac{\partial E_n}{\partial z_i}$ which we will now turn to.

6.3.1 The ouput layer

For the output layer $\frac{\partial E_n}{\partial z_i}$ can be directly calculated. To see this let the loss function be the sum of squared errors

$$\begin{aligned} E = \frac{1}{2}\sum_{n=1}^{N} ||f(\mathbf{x}_n;\mathbf{w}) - \mathbf{y}_n||^2 &= \frac{1}{2}\sum_{n=1}^{N}\left(\sqrt{\sum_{k=1}^{K}\left(\left[f(\mathbf{x}_n;\mathbf{w})\right]_k - \left[\mathbf{y}_n\right]_k\right)^2}\right)^2 \\ &= \frac{1}{2}\sum_{n=1}^{N}\sum_{k=1}^{K}\left(\left[f(\mathbf{x}_n;\mathbf{w})\right]_k - \left[\mathbf{y}_n\right]_k\right)^2 \\ &= \frac{1}{2}\sum_{n=1}^{N}\sum_{k=1}^{K}\left(\left[\mathbf{z}_n\right]_k - \left[\mathbf{y}_n\right]_k\right)^2 \end{aligned}$$

for $k \in V_{output}$. Therefore we have (for $i \in V_{output}$ and $(j,i) \in \mathcal{A}$)

$$\begin{aligned} \frac{\partial E_n}{\partial z_i} &= \frac{\partial}{\partial z_i}\frac{1}{2}\sum_{k=1}^{K}\left(\left[\mathbf{z}_n\right]_k - \left[\mathbf{y}_n\right]_k\right)^2 \\ &= \left[\mathbf{z}_n\right]_i - \left[\mathbf{y}_n\right]_i \end{aligned}$$

For $h_i(a_i) = a_i \implies h_i'(a_i) = 1$ where $i \in V_{output}$ we thus have

$$\begin{aligned}\frac{\partial E}{\partial w_{ij}} &= \sum_{n=1}^{N} \frac{\partial E_n}{\partial z_i} h_i'(a_i) z_j \\ &= \sum_{n=1}^{N} \frac{\partial E_n}{\partial z_i} \left[\mathbf{z}_n\right]_j \\ &= \sum_{n=1}^{N} \left(\left[\mathbf{z}_n\right]_i - \left[\mathbf{y}_n\right]_i\right) \left[\mathbf{z}_n\right]_j\end{aligned}$$

6.3.2 Hidden layers

For hidden layers we apply the apply the chainrule to obtain

$$\begin{aligned}\frac{\partial E}{\partial z_j} &= \sum_{k \text{ in } (j,k) \in \mathcal{A}} \frac{\partial E}{\partial z_k} \frac{\partial z_k}{\partial z_j} \\ &= \sum_{k \text{ in } (j,k) \in \mathcal{A}} \frac{\partial E}{\partial z_k} \frac{\partial z_k}{\partial a_k} \frac{\partial a_k}{\partial z_j} \\ &= \sum_{k \text{ in } (j,k) \in \mathcal{A}} \frac{\partial E}{\partial z_k} h_k'(a_k) w_{kj}.\end{aligned}$$

The partial derivatives of the layer just before V_{ouput} is then calculated by pluging in $\frac{\partial E}{\partial z_k}$, $k \in V_{output}$ and $(j,k) \in \mathcal{A}$. Then the third layers uses the second layers calculated $\frac{\partial E}{\partial z_j}$ and so on. This is sometimes referred to as the backwards pass.

7 Deep Learning

7.1 Convolution / cross-correlation

A convolution is a filtering transformation applied to image-like data. Image-like in the sence that datapoints close to each other in the dataset structure somehow relate to each other - like two pixels are closely related in a picture.

Formally let $C(i,j)$ be the output image from applying a convolution operation (or cross correlation operation) to the pixel in image I at location (i,j).

Additionally C takes a kernel matrix K as input.

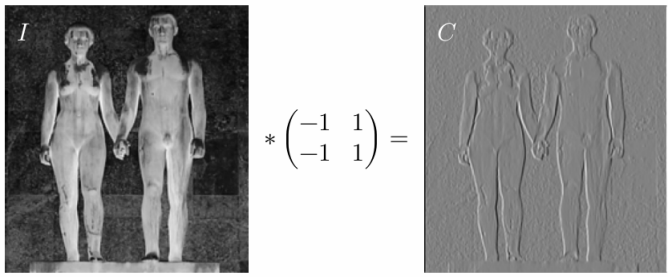
Cross correlation: It is intuitive and defined as

$$C_{corr}(i,j) = \sum_{m \in \mathcal{M}} \sum_{n \in \mathcal{N}} K(m,n) \cdot I(i+m, j+n),$$

where the kernel for example could be the vertical edge *filter kernel*:

$$K = \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix},$$

and thus $\mathcal{M} = \{0,1\}$ and $\mathcal{N} = \{0,1\}$. So basically for each pixel we overlay the filter kernel K, weigh the surronding pixels accordingly and sum. An example is given here:



$$* \begin{pmatrix} -1 & 1 \\ -1 & 1 \end{pmatrix} =$$

Convolution: Essentially the same method but where we first flip K above 180-degrees, shiftet so the bottom right is at the pixel (i,j) and then apply cross correlation. The formula is

$$C_{conv}(i,j) = (I * K)(i,j) = \sum_{m \in \mathcal{M}} \sum_{n \in \mathcal{N}} K(m,n) \cdot I(i-m, j-n).$$

Note the shift dependents on $\mathcal{M}$ and $\mathcal{N}$. For example there is no shift for symmetrical convolutions

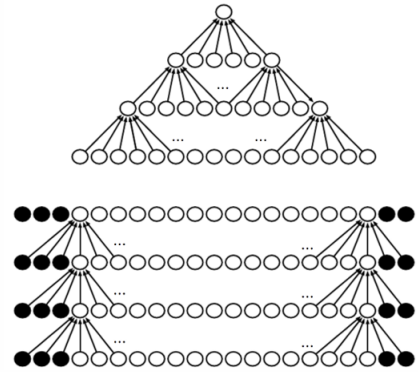
$$\mathcal{M} = \mathcal{N} = \{-k, -(k-1), \ldots, 0, \ldots, (k-1), k\}.$$

(Remember this is all in comparison with cross-correlation).

7.1.1 *Padding*

Convolution layers or cross correlation layers reduce the number of pixels. For example a 3 by 3 cross correlation will remove 2 pixels in each dimension thus reducing an image from (M, N) to $(M-2, N-2)$ to avoid this one can add *padding* around the input image I so that it becomes $(M+2, N+2)$.

The padding values are usually just zeros (zero padding) since the specific values rarely matters in practice. But it could also be some notion of mean values.



Goodfellow, Bengio, Courville. *Deep Learning*, 2016

0	0	0	0	0	0
0	X_{11}	X_{12}	X_{13}	X_{14}	0
0	X_{21}	X_{22}	X_{23}	X_{24}	0
0	X_{31}	X_{32}	X_{33}	X_{34}	0
0	X_{41}	X_{42}	X_{43}	X_{44}	0
0	0	0	0	0	0

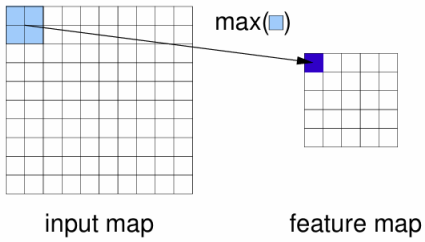
Bishop, Bishop. *Deep Learning*, 2024

7.1.2 *Equivariance*

If the input (for example a traffic sign in a picture) is translated / moved then the output (i.e. the feature space of the neurons in a particular layer) will translate in the same way. This is why convolutional neural networks are particularly relevant for image recognition tasks since it usually does not matter where in the frame a person or item is located.

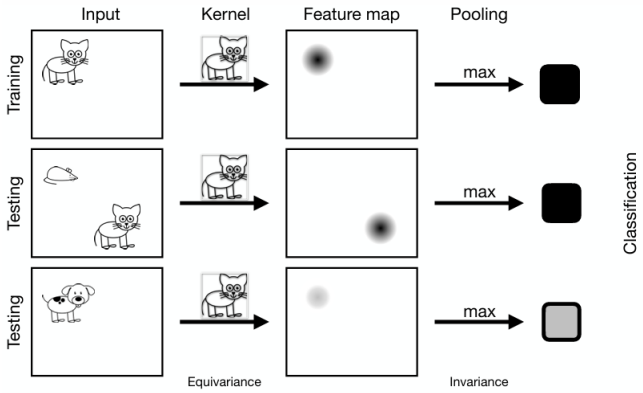
7.2 Pooling layer

Pooling layers reduce dimensionality by for example looping over all non-intersecting 2×2 grids on the feature image with a *stride* of 2. Then applying the $\max\{\cdot\}$ operator over those four values as the output. Thus in this example a 4 by 4 feature map is reduces to 2 by 2.



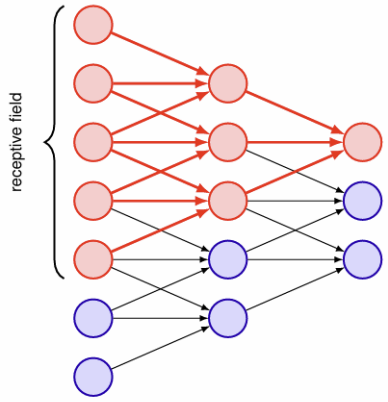
7.2.1 *Invariance*

Invariance refers to operations that does not change its output when the input is translated / shiftet. Pooling layers can help with invariance which can be understood through an example. Let the kernel detect cat-shapes. Then translating the cat 5 pixels to the right translates the feature map equivaraintly 5 pixels to the right (remember that the feature space is the output of the cross-correlation using the cat-kernel). Applying a max pooling layer will still just indicate `True` there is a cat. Thus the algorithm is *invariant* to translations of the cat.



7.2.2 *Receptive field*

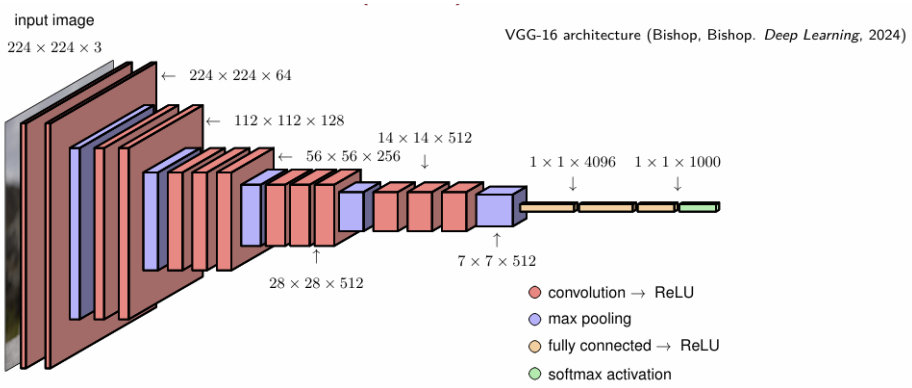
Nerons in deeper layers have a greater *effective receptive* field than its kernel would suggest. This is simply because the feature space the neuron percieves comes from a poolin layer which have condensed larger field.



Bishop, Bishop. *Deep Learning*, 2024

7.3 Example: VGG-16 architecture

As a quick recap I will explain the general details of the VGG-16 architecture depicted here.



The input layer is one RGB image - so there is 3 input channels corresponding to red, green and blue respectively. Each input channel is of size 244×244. The convolutional layer outputs 64 feature maps i.e. there are 64 kernels and biases needing to be trained in this single layer. Between each layer the non-linear ReLU activation function is applied. Each 224×224 feature-map is then reduced by max pooling to 122×122 and convolutional layers now take the 64 features and double them and creating 128 feature maps (thus and additional 128 kernels and biases needs training for each layer).

In the end, we obtain 512 feature maps, each of size 7×7. These are flattened

into a single vector containing $7 \cdot 7 \cdot 512 = 25,088$ features, which serves as input to a fully connected layer with 4096 neurons. This is followed by another fully connected layer with 4096 neurons and finally an output layer with 1,000 neurons, corresponding to the 1000 possible classification categories. A softmax activation is applied to the final output to obtain a probability distribution over these categories.

7.4 Tricks-of-the-Trade

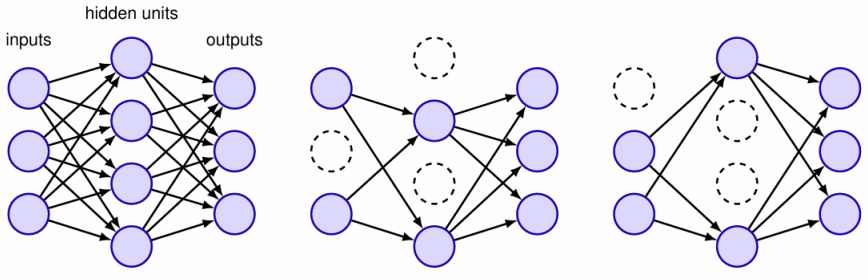
7.4.1 Ensemble Methods

Ensemble methods combat overfitting through the *cancellation of errors effect* and has proven to be very succesfull in the CNN setting. CNN ensembles can be created over the same datasets because of the importance of random initialization on the local minimum obtained.

Reminder: cancellation of errors effect: When individual ensemble members perform better than random guessing and make independent errors, their errors average out and the combined model outperforms the individual predictors.

7.4.2 Dropout

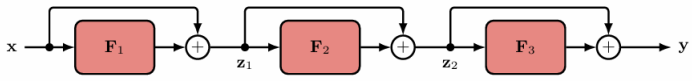
A heuristic method for avoiding overfitting is *dropout*. It is an ensemble method where each member of the ensemble is a subnetwork where randomly a binary mask **R** has been applied to remove neurons. Then training commences on all the subnetworks individually to create the ensemble. The two right models in the illustration would be two members of the whole ensemble.



Bishop, Bishop. *Deep Learning*, 2024

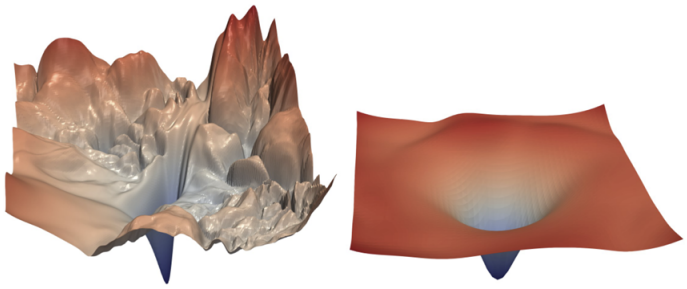
7.4.3 *Residual Leraning / Shorcuts*

Yet another method to combat local minima is shortcuts. Where in each layer the input data directly passes trough as illustrated below.



Bishop, Bishop. *Deep Learning*, 2024

This can drastically smooth the error surface. This example visualizes the error surface (loss vs. parameters) for a network with 56 layers without (a) and with (b) shortcuts.



(a) (b)

Li, Xu, Taylor, Studer, Goldstein. Visualizing the Loss Landscape of Neural Nets, *NeurIPS*, 2018

7.4.4 *Augmentation og Invariance*

Data augmentation refers to applying transformations to the input data that do not change their labels. Let $T = \{T1, \ldots, \}$ be a set of transformations. During training, for each

element added to a mini-batch it is decided if and which transformations are applied to it. In this way, we learn invariance with respect to the transformations in T. If augmentation is always applied, the network hardly see exactly the same input twice. Note that the "new" training examples are not i.i.d. samples!

Examples of transformations for images could be cropping, rotating, flipping, blurring, adding noise, ...

7.4.5 Batch and Layer Normalization

Batch normalization and layer normalization normalize intermediate neuron activations across different dimensions. This can stabilize gradient propagation, improve the conditioning of the optimization problem, and allow gradient descent to converge more reliably and often faster.

- Batch normalization normalizes *each* neurons activation over *multiple* observations (i.e. the batch).
- Layer normalization normalizes neuron activation *layer* for *each* observation.

III | Unsupervised Learning

PCA, *k*-means, *k*-medoids, *k*-medians, SNE, and t-SNE

8 Clustering

8.1 General definitions

Clustering is the task of grouping unlabeled observations according to similarity. The idea is that observations in the same cluster should be similar, while observations in different clusters should be dissimilar. The first question is therefore what we mean by *similar*.

For example, the slides group the same collection of footwear in two ways. Grouping by type gives three clusters, whereas grouping by color gives four. Both groupings make sense given their respective definitions of similarity. In a dataset with numerical features, the same issue arises through the choice and scale of the features. If we multiply one coordinate by ten, its contribution to squared Euclidean distance becomes one hundred times as large.

Clustering is a form of *unsupervised learning*. We observe $\mathbf{x}_1, \ldots, \mathbf{x}_N$ without target labels $y_1, \ldots, y_N$ that tell us how to group them. Clustering can help us explore the structure of the data, summarize many observations by a few representatives, or identify observations that lie far from the main groups.

8.1.1 Mathematical setting

Let $X = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$ be the dataset. We need some way of comparing observations, for example a dissimilarity

$$d \colon X \times X \to [0, \infty)$$

where smaller values mean more similar, or a similarity $s \colon X \times X \to [0, 1]$ where larger values mean more similar. In the methods below, the observations are vectors $\mathbf{x}_n \in \mathbb{R}^m$.

A *partition* of X into k clusters $\{C_1, \ldots, C_k\}$ satisfies

$$\bigcup_{j=1}^{k} C_j = X, \qquad C_i \cap C_j = \emptyset \quad \text{for } i \neq j, \qquad C_j \neq \emptyset \quad \text{for all } j.$$

I.e. every observation belongs to exactly one cluster. The numbers $1, \dots, k$ are just names for the clusters. Exchanging the names of two clusters gives the same partition.

A distance tells us how to compare two observations. To compare two *partitions*, we need a criterion for the quality of the whole grouping. The main approach in this lecture is to define a cost and look for a partition with small cost. Here k is given to the algorithm, even if we have to choose it ourselves before running it.

8.2 The k-means problem

8.2.1 The objective function

Suppose we have k centers $C = \{\mathbf{c}_1, \dots, \mathbf{c}_k\} \subset \mathbb{R}^m$. Each observation is represented by its closest center. The cost of this representation is

$$\phi(X, C) \equiv \sum_{n=1}^{N} \min_{j \in \{1,\dots,k\}} \|\mathbf{x}_n - \mathbf{c}_j\|_2^2. \tag{8.1}$$

For a given point we first find its closest center and then add its squared distance to that center. Squaring the Euclidean distance preserves the ordering of the centers for a given point. It changes how much near and far points contribute to the total cost.

The k-means *problem* is to find the best possible k centers

$$\phi^\star = \min_{\substack{C \subset \mathbb{R}^m \\ |C|=k}} \phi(X, C).$$

The centers can be anywhere in $\mathbb{R}^m$, including locations between observed points. The exact optimization problem is NP-hard in general. Lloyd's algorithm, which we discuss below, is a way to search for a low-cost solution.

Sidenote. This objective is also a reconstruction error, as in PCA. Here the encoding of an observation is a cluster index j, and its reconstruction is the corresponding center $\mathbf{c}_j$. Dividing (8.1) by N gives the average squared reconstruction error and leaves the minimizing centers the same.

8.2.2 *Special case $k = 1$ and why the mean appears*

With one center, all observations are in the same cluster. More generally, let S be any nonempty set of observations and define its centroid by

$$\bar{\mathbf{x}}_S = \frac{1}{|S|} \sum_{\mathbf{x} \in S} \mathbf{x}.$$

Statement. The centroid is the unique minimizer of $\sum_{\mathbf{x} \in S} \|\mathbf{x} - \mathbf{c}\|_2^2$ over $\mathbf{c} \in \mathbb{R}^m$.

Proof. Write $\mathbf{x} - \mathbf{c} = (\mathbf{x} - \bar{\mathbf{x}}_S) + (\bar{\mathbf{x}}_S - \mathbf{c})$. Using the inner product to expand the square gives

$$\begin{aligned} \sum_{\mathbf{x} \in S} \|\mathbf{x} - \mathbf{c}\|_2^2 &= \sum_{\mathbf{x} \in S} \|\mathbf{x} - \bar{\mathbf{x}}_S\|_2^2 \\ &\quad + 2 \underbrace{\left(\sum_{\mathbf{x} \in S} (\mathbf{x} - \bar{\mathbf{x}}_S) \right)'}_{= \mathbf{0}'} (\bar{\mathbf{x}}_S - \mathbf{c}) \\ &\quad + |S| \|\bar{\mathbf{x}}_S - \mathbf{c}\|_2^2 \\ &= \sum_{\mathbf{x} \in S} \|\mathbf{x} - \bar{\mathbf{x}}_S\|_2^2 + |S| \|\bar{\mathbf{x}}_S - \mathbf{c}\|_2^2. \end{aligned}$$

The first term is fixed. The second term is nonnegative and is zero exactly when $\mathbf{c} = \bar{\mathbf{x}}_S$. Thus the cost of using any other center is the cost at the mean plus a penalty proportional to the squared distance from the mean.

Example. For the four observations $(-6, 0)$, $(0, -1)$, $(2, 3)$, and $(5, 0)$, we take the mean separately along each coordinate

$$\bar{\mathbf{x}} = \left(\frac{-6 + 0 + 2 + 5}{4}, \frac{0 - 1 + 3 + 0}{4} \right) = \left(\frac{1}{4}, \frac{1}{2} \right).$$

The optimal center lies between the observations.

8.3 The k-means algorithm (Lloyd's algorithm)

8.3.1 Alternating between assignments and centers

The problem has two sets of unknowns. We need to decide which observations belong together and where to put their centers. Each of these is easy to optimize once we fix the other. To see this, let $z_n \in \{1, \dots, k\}$ be the cluster index of observation n and write the cost for a given assignment as

$$J(\mathbf{z}, C) = \sum_{n=1}^{N} \|\mathbf{x}_n - \mathbf{c}_{z_n}\|_2^2.$$

For fixed centers, $\phi(X, C) = \min_{\mathbf{z}} J(\mathbf{z}, C)$.

1. **Initialize.** Choose k distinct centers $\mathbf{c}_1, \dots, \mathbf{c}_k$. For example, choose k observations at random as the initial centers.
2. **Assign points to centers.** With the centers fixed, choose for each observation

   $$z_n \in \operatorname*{arg\,min}_{j \in \{1,\dots,k\}} \|\mathbf{x}_n - \mathbf{c}_j\|_2^2.$$

   This minimizes J over the assignments because each summand depends on only one z_n. The resulting clusters are $C_j = \{\mathbf{x}_n : z_n = j\}$.
3. **Update the centers.** With the assignments fixed, the objective separates into k terms, one for each cluster

   $$J(\mathbf{z}, C) = \sum_{j=1}^{k} \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \mathbf{c}_j\|_2^2.$$

   By the result for $k = 1$, each nonempty cluster gets the update

   $$\mathbf{c}_j \leftarrow \frac{1}{|C_j|} \sum_{\mathbf{x} \in C_j} \mathbf{x}.$$

4. Repeat the assignment and update steps until the assignments stop changing. Return the assignments and the final centers.

Two details matter when spelling out the algorithm. If several centers are equally

close, keep the previous assignment if it is among the minimizers and otherwise choose the smallest minimizing index. If a cluster is empty, its mean is undefined. One convention is to keep its current center for that update, in which case there can be fewer than k nonempty groups at the end. Implementations can also reseed empty clusters when k nonempty groups are needed.

8.3.2 Why the algorithm converges

Statement. With the tie and empty-cluster conventions above, Lloyd's algorithm reaches a fixed assignment after finitely many iterations.

Proof. Let $\mathbf{z}^{(t)}$ and $C^{(t)}$ denote the assignments and centers after the tth update. The next two steps satisfy

$$\begin{aligned} J(\mathbf{z}^{(t)}, C^{(t)}) &\geq J(\mathbf{z}^{(t+1)}, C^{(t)}) && \text{(assign to closest centers)} \\ &\geq J(\mathbf{z}^{(t+1)}, C^{(t+1)}) && \text{(update to the means).} \end{aligned}$$

If an observation changes cluster, its new center must be strictly closer under our tie rule. Thus the first inequality is strict whenever the assignment changes. After the update, each nonempty cluster's contribution to the cost is determined by its members and their mean. Empty clusters contribute zero. There are at most k^N assignments and hence finitely many possible costs after the updates. We can therefore have only finitely many strict decreases. Once the assignments stay the same, the center update also leaves the centers unchanged.

The convergence result tells us that the algorithm reaches a clustering that is stable under its two updates. Its cost can still be far above $\phi^\star$. The algorithm is simple, and an iteration takes $O(Nkm)$ operations with a direct distance calculation. It often stabilizes quickly in practice. Its result can depend strongly on the initialization, the choice of k, and the presence of outliers.

8.3.3 A four-point example with arbitrarily bad cost

Statement. For any $\alpha > 1$, there is a dataset and an initialization for which the final cost of Lloyd's algorithm is at least $\alpha\phi^\star$.

Proof by construction. Consider four points on the real line, ordered as

$$p_1 \overset{y}{\longleftrightarrow} p_2 \overset{z}{\longleftrightarrow} p_3 \overset{x}{\longleftrightarrow} p_4, \qquad 0 < x < y < z.$$

Here x, y, z denote gaps between consecutive points. Let $k = 3$. At least two of the four points must share a center. For two points at distance a, the best shared center is their midpoint and the cost of those two points is

$$2\left(\frac{a}{2}\right)^2 = \frac{a^2}{2}.$$

The closest pair is $\{p_3, p_4\}$, at distance x. Any shared pair therefore costs at least $x^2/2$. Putting one center at p_1, one at p_2, and one at the midpoint of p_3 and p_4 attains that lower bound. Thus $\phi^\star = x^2/2$.

Now initialize the centers at p_1, p_3, p_4. The point p_2 joins p_1 since $y < z$, and each of the other points goes to its own center. The update moves the first center to $(p_1 + p_2)/2$. Both p_1 and p_2 are now $y/2$ away from that center and farther away from the other centers. The next assignment is therefore the same. The final cost and its ratio to the optimum are

$$\phi = \frac{y^2}{2}, \qquad \frac{\phi}{\phi^\star} = \left(\frac{y}{x}\right)^2.$$

We can make the ratio as large as we like by choosing y/x large and taking $z > y$. For example, gaps $(y, z, x) = (10, 11, 1)$ give cost 50 compared with the optimal cost $1/2$.

8.3.4 Class exercise with three rows and three columns

To put numbers on the figure, take the nine points with horizontal coordinates in $\{-4, 0, 4\}$ and vertical coordinates in $\{-1, 0, 1\}$. The vertical gaps are smaller, so the three columns are a natural choice of clusters.

If the initial centers are the three points in the middle column, each observation is closest to the center on its own row. The horizontal distance to the three centers is the same, and the center on its own row has zero vertical distance. The mean of each row is already its middle point. Thus the row partition is a fixed

point with cost

$$\phi_{\text{rows}} = 3(4^2 + 0^2 + 4^2) = 96.$$

If the initial centers are the three points in the middle row, each observation goes to its own column. The means are again unchanged, and the cost is

$$\phi_{\text{columns}} = 3(1^2 + 0^2 + 1^2) = 6.$$

In this numerical example the column partition is optimal. Any cluster containing points from two different columns must contain a pair with horizontal separation at least 4. Those two points alone cost at least $4^2/2 = 8$ when they share a center. So an optimal clustering keeps the three columns separate. With three centers available, it uses one center for each column.

8.4 Choosing the initial centers

One way to deal with sensitivity to initialization is to run Lloyd's algorithm several times and keep the result with the smallest ϕ. Another way is to make the initialization depend on where we have already placed centers.

Let B be the set of centers chosen so far and define

$$D(\mathbf{x}, B) \equiv \min_{\mathbf{c} \in B} \|\mathbf{x} - \mathbf{c}\|_2.$$

Farthest traversal chooses the next center at a point maximizing $D(\mathbf{x}, B)$. The k-means++ method uses these distances to give points different sampling probabilities.

8.4.1 k-means++

1. Choose $\mathbf{c}_1$ uniformly at random from X.
2. For $i = 2, \ldots, k$, let $B_{i-1} = \{\mathbf{c}_1, \ldots, \mathbf{c}_{i-1}\}$ and sample the next center according to
$$\mathbb{P}(\mathbf{c}_i = \mathbf{x} \mid B_{i-1}) = \frac{D(\mathbf{x}, B_{i-1})^2}{\sum_{\mathbf{y} \in X} D(\mathbf{y}, B_{i-1})^2}.$$
3. Run Lloyd's algorithm with the resulting k centers.

The denominator is the current k-means cost with the centers chosen so far. The numerator is the contribution of $\mathbf{x}$ to that cost. Thus a point with twice the distance to its nearest center has four times the sampling weight. A point that is already a center has weight zero. After adding a center we recompute the distances, so points near the new center get less weight in the next draw. This is why the sampling is called *adaptive*.

8.4.2 Worked example of k-means++

Consider $X = \{(5,0),(0,1),(0,-1),(-5,0)\}$ and let $k = 3$. Each point has probability $1/4$ of being the first center. Suppose that $\mathbf{c}_1 = (5,0)$ is chosen. The squared distances to this center are

$\mathbf{x}$	$(5,0)$	$(0,1)$	$(0,-1)$	$(-5,0)$
$D(\mathbf{x},\{\mathbf{c}_1\})^2$	0	26	26	100

because, for example, $\|(0,1)-(5,0)\|_2^2 = 5^2 + 1^2 = 26$. The weights sum to 152. The probabilities of choosing $(0,1)$, $(0,-1)$, and $(-5,0)$ as the second center are $26/152$, $26/152$, and $100/152$, respectively.

Now suppose $\mathbf{c}_2 = (0,1)$. The two remaining points have squared distances to their nearest selected center given by

$$D\big((0,-1),\{\mathbf{c}_1,\mathbf{c}_2\}\big)^2 = \min\{26,4\} = 4,$$
$$D\big((-5,0),\{\mathbf{c}_1,\mathbf{c}_2\}\big)^2 = \min\{100,26\} = 26.$$

Hence the probabilities for the third center are $4/30$ for $(0,-1)$ and $26/30$ for $(-5,0)$.

Suppose $\mathbf{c}_3 = (-5,0)$ is chosen. In the first Lloyd assignment, $(0,-1)$ joins $(0,1)$ because its squared distance to that center is 4, compared with 26 to each of the other centers. The two outer points are singleton clusters. The center update moves the middle center to

$$\frac{(0,1)+(0,-1)}{2} = (0,0).$$

The next assignment stays the same. The singleton clusters cost zero and each

of the two middle points costs 1, so the final cost is 2. This also happens to be optimal, since the closest pair is at distance 2 and at least one pair must share a center.

8.4.3 Guarantee in expectation

Statement (Arthur and Vassilvitskii, 2007). The expected cost of the centers selected by k-means++ satisfies

$$\mathbb{E}[\phi(X, C_{\text{seed}})] \leq 8(\ln k + 2)\phi^{\star}.$$

The expectation is over the random choice of seeds for a fixed dataset. Individual runs can produce different costs. Since each Lloyd iteration can only decrease the cost, the same upper bound holds for the expected final cost. Thus k-means++ gives an $O(\log k)$ approximation in expectation for $k \geq 2$.

8.5 Choosing the number of clusters

Let $\phi_k^{\star}$ be the smallest possible cost with k centers. Adding an extra center leaves the old centers available and can only reduce a point's distance to its nearest center. Therefore

$$\phi_{k+1}^{\star} \leq \phi_k^{\star}.$$

In particular, with one center per observation the cost is zero. So minimizing the cost over k alone will favor using as many centers as we allow, even if the goal was to summarize the data.

The *elbow method* fits clusterings for several values of k and plots the resulting cost against k. The plot is sometimes called a scree plot. Look for a bend where the cost goes from falling sharply to falling more slowly. In the slide example, the drop from $k = 2$ to $k = 3$ is much larger than the later drops. This suggests that three centers capture much of the structure and that extra centers give smaller improvements. When constructing the plot from Lloyd runs, using several initializations per k helps reduce the effect of one poor run on the curve.

The elbow is a heuristic. Some datasets give a gradual curve with several plausible choices. Domain knowledge may suggest how many groups to expect, and

the intended use may call for broader or more detailed groups. For example, a three-cluster summary can capture the major groups even if a more detailed analysis would split one of them further.

8.6 Beyond k-means

8.6.1 k-medoids and the choice of representative

Suppose the observations represent people. The centroid describes an average feature vector, which may correspond to no actual person. If we want an observed person to represent each group, we can restrict the centers to observations in X. These centers are called *medoids*. With the squared Euclidean cost used in the slides, the problem becomes

$$\min_{\substack{C \subset X \\ |C|=k}} \sum_{\mathbf{x} \in X} \min_{\mathbf{c} \in C} \|\mathbf{x} - \mathbf{c}\|_2^2.$$

Relative to k-means, we have changed the set of feasible centers. For example, with the one-dimensional cluster $S = \{0, 2\}$, the centroid is 1 with cost 2. The feasible medoids are 0 and 2, each with cost 4.

For squared Euclidean distance and a fixed cluster, the centroid identity gives an easy way to understand the medoid. Among the points in the cluster, choose one closest to the centroid. This minimizes $|S|\|\bar{\mathbf{x}}_S - \mathbf{c}\|_2^2$, which is the only part of the within-cluster cost that depends on $\mathbf{c}$. More generally, k-medoids can be formulated using a pairwise dissimilarity between observations. We can then compare candidate representatives directly, even when taking an average of the observations has no clear meaning.

One exact solution is to evaluate all $\binom{N}{k}$ possible sets of medoids and take the set with lowest cost. This is polynomial in N for fixed k, but the optimization problem is NP-hard when k is part of the input.

8.6.2 Outliers and why we might prefer the median

An outlier is an observation that deviates substantially from the rest of the data. It might be a measurement error, a recording error, or a valid but rare event. In

k-means, an observation at distance r from its closest center contributes r^2 to the cost. This can pull a centroid toward the outlier. It can also make it worth spending a whole center on that one observation. In the slide with three dense groups and two isolated points, a solution with $k = 3$ can consequently devote a center to an isolated point and leave two dense groups sharing one center.

The difference between the mean and median is easy to see in one dimension. The sample $(0, 1, 2)$ has mean and median both equal to 1. Adding 101 gives

$$\text{mean} = \frac{0+1+2+101}{4} = 26, \qquad \text{median} = \frac{1+2}{2} = \frac{3}{2}.$$

The mean responds to how far away 101 is. The median depends on the positions of the observations in the sorted sample, so making the largest observation even larger leaves the median unchanged.

8.6.3 Why the median minimizes absolute deviations

Let $x_{(1)} \leq \cdots \leq x_{(n)}$ be the sorted observations of a nonempty one-dimensional sample. The sample median is

$$\text{median}(S) = \begin{cases} x_{((n+1)/2)}, & n \text{ odd}, \\ \dfrac{x_{(n/2)} + x_{(n/2+1)}}{2}, & n \text{ even}. \end{cases}$$

Statement. The median minimizes $\sum_{i=1}^{n} |x_i - c|$ over $c \in \mathbb{R}$.

Proof. Pair the smallest observation with the largest, the second smallest with the second largest, and so on. For any pair $a \leq b$,

$$|a - c| + |b - c| \geq b - a,$$

with equality whenever $c \in [a, b]$. Thus we minimize a pair's contribution by placing c between its two members. If n is even, the pair intervals are nested, with the narrowest interval given by $[x_{(n/2)}, x_{(n/2+1)}]$. Every point in this interval minimizes the sum. Taking its midpoint gives the usual sample median. If n is odd, the unpaired middle observation adds the term $|x_{((n+1)/2)} - c|$. Choosing

$c = x_{((n+1)/2)}$ makes this term zero and also minimizes all the paired terms.

For vectors, the ℓ_1 norm is the sum of absolute differences along the coordinates. Thus the within-cluster cost separates as

$$\sum_{\mathbf{x}\in S} \|\mathbf{x} - \mathbf{c}\|_1 = \sum_{\ell=1}^{m} \sum_{\mathbf{x}\in S} |x_\ell - c_\ell|.$$

Each c_ℓ can be chosen independently as the median of the ℓth coordinates. This is the *coordinate-wise median*.

8.6.4 The k-medians problem and algorithm

The k-medians objective uses the ℓ_1 distance

$$\phi_{\text{median}}(X, C) = \sum_{\mathbf{x}\in X} \min_{\mathbf{c}\in C} \|\mathbf{x} - \mathbf{c}\|_1, \qquad \min_{\substack{C\subset\mathbb{R}^m \\ |C|=k}} \phi_{\text{median}}(X, C).$$

The corresponding algorithm alternates between two steps.

1. **Assign.** Give each observation the index of its closest center under ℓ_1 distance.
2. **Update.** Set the center of each nonempty cluster to its coordinate-wise median.

The two steps minimize the same objective over assignments and centers, respectively. The assignment step uses ℓ_1 because that is the distance in the objective. The update uses medians because they minimize the resulting sum of absolute deviations. A distant point's contribution grows linearly with its ℓ_1 distance. Outliers can still affect the assignments or attract a center.

8.7 Flat versus hierarchical clustering

Flat or partitional clustering. The output is a single partition. For example, k-means, k-medoids, and k-medians each return a grouping at the selected value of k. The slides also list spectral clustering as a partitional approach.

Hierarchical clustering. The output is a nested collection of groups, which can be represented as a tree called a *dendrogram*. A cut through the tree gives a flat

partition at a chosen level of detail. There are two ways to build the hierarchy.

- **Agglomerative.** Start with each observation in its own cluster and repeatedly merge clusters. We need a rule for comparing whole clusters to decide which ones to merge.
- **Divisive.** Start with all observations in one cluster and repeatedly split clusters into smaller groups. We need a rule for deciding where to split.

For example, a dataset might have an upper group and a lower group, with two subgroups inside the lower group. A hierarchy can record both the coarse partition into two groups and the finer partition into three. The difference is which level of the hierarchy we use.

8.8 Conceptual issues

8.8.1 Similarity depends on the question

There is often no ground-truth partition available to settle which clustering is correct. A collection of points can have a few recognizable groups at a broad scale and several subgroups at a finer scale. The footwear example shows a different source of ambiguity. Changing the feature we care about, from type to color, changes the grouping even at the same scale. A clustering can therefore be interpreted in terms of the features, distance, and objective that produced it.

8.8.2 Cluster membership is transitive

For a partition, let $\mathbf{u} \sim \mathbf{v}$ mean that $\mathbf{u}$ and $\mathbf{v}$ are in the same cluster. This is an equivalence relation. It is reflexive because every point shares a cluster with itself. It is symmetric because sharing a cluster works in both directions. It is transitive because

$$\mathbf{u} \sim \mathbf{v} \quad \text{and} \quad \mathbf{v} \sim \mathbf{w} \quad \Longrightarrow \quad \mathbf{u} \sim \mathbf{w}.$$

A relation defined by pairwise similarity can fail transitivity. For example, on the real line define two points to be similar if they are at distance at most 1. Then 0 is similar to 1 and 1 is similar to 2, but 0 and 2 are at distance 2. A partition has

to resolve this conflict. If it puts both nearby pairs in the same cluster, it must also put 0 and 2 together. If it separates the endpoints, it must separate at least one of the nearby pairs. The clustering objective or merge rule determines how such conflicts are resolved.

9 Principal Component Analysis (linear Dimensionality Reduction)

9.1 Definitions

9.1.1 Dimensions

- N is the number of observations.
- D is the dimensionality of the original observations $\mathbf{x}_i \in \mathbb{R}^D$. For example, flattening a 92×112 grayscale image gives a vector with $D = 92 \cdot 112 = 10304$ entries.
- $d < D$ is the dimensionality of the reduced representation $\mathbf{z}_i \in \mathbb{R}^d$.
- $X \in \mathbb{R}^{N \times D}$ is the data matrix. Its ith row is $\mathbf{x}_i'$, just as observations are stacked in the design matrix in linear regression.
- $U_d = [\mathbf{u}_1, \ldots, \mathbf{u}_d] \in \mathbb{R}^{D \times d}$ contains the d directions we keep. Its columns are orthonormal, so $U_d' U_d = I_d$.

PCA is an unsupervised method for dimensionality reduction. We want to describe each observation with fewer numbers while preserving as much of the original variation as possible. This is useful for compression, feature extraction, and visualization. For example, if $d = 2$, we can plot the reduced observations in a scatter plot.

9.2 The objective of PCA

Given a low-dimensional representation $\mathbf{z}_i$, reconstruct the original observation using the decoder

$$f_{\text{dec}}(\mathbf{z}_i) = \mathbf{b} + U_d \mathbf{z}_i, \qquad \mathbf{b} \in \mathbb{R}^D.$$

The vector $\mathbf{b}$ is the starting point of the reconstruction. The entries of $\mathbf{z}_i$ tell us how far to move in each of the d directions in U_d. The possible reconstructions therefore lie in the affine subspace $\mathbf{b} + \text{span}(\mathbf{u}_1, \ldots, \mathbf{u}_d)$.

We measure the quality of the reconstruction by the average squared distance between the original and reconstructed observations

$$J = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{x}_i - f_{\text{dec}}(\mathbf{z}_i)\|_2^2.$$

The optimization problem is

$$\min_{\substack{\mathbf{b},\, U_d,\, \mathbf{z}_1,\ldots,\mathbf{z}_N \\ U_d' U_d = I_d}} \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{x}_i - \mathbf{b} - U_d \mathbf{z}_i\|_2^2.$$

We can work through this in three steps. First find the best encoding for fixed $\mathbf{b}$ and U_d. Then choose $\mathbf{b}$. Finally find the directions in U_d.

9.3 Optimal reconstruction

9.3.1 Changing basis and keeping fewer directions

Let $U = [\mathbf{u}_1, \ldots, \mathbf{u}_D]$ be a full orthonormal basis of $\mathbb{R}^D$, so

$$\mathbf{u}_k' \mathbf{u}_j = \begin{cases} 1, & k = j, \\ 0, & k \neq j. \end{cases}$$

For a fixed starting point $\mathbf{b}$ we can express the displacement of $\mathbf{x}_i$ from $\mathbf{b}$ as

$$\mathbf{x}_i - \mathbf{b} = \sum_{j=1}^{D} a_{ij} \mathbf{u}_j.$$

To find a_{ik}, premultiply by $\mathbf{u}_k'$:

$$\begin{aligned}\mathbf{u}_k'(\mathbf{x}_i - \mathbf{b}) &= \sum_{j=1}^{D} a_{ij}\mathbf{u}_k'\mathbf{u}_j \\ &= a_{ik}\underbrace{\mathbf{u}_k'\mathbf{u}_k}_{=1} = a_{ik}.\end{aligned}$$

Thus the exact change of basis uses all D terms

$$\mathbf{x}_i - \mathbf{b} = \sum_{j=1}^{D} \big(\mathbf{u}_j'(\mathbf{x}_i - \mathbf{b})\big)\mathbf{u}_j.$$

If we keep only the first d directions, we approximate the displacement by the first d terms of this sum. Why are these still the best coefficients after dropping the other directions? Write z_{ij} for the jth entry of $\mathbf{z}_i$. By orthonormality,

$$\begin{aligned}\|\mathbf{x}_i - \mathbf{b} - U_d\mathbf{z}_i\|_2^2 &= \left\| \sum_{j=1}^{d}(a_{ij} - z_{ij})\mathbf{u}_j + \sum_{j=d+1}^{D} a_{ij}\mathbf{u}_j \right\|_2^2 \\ &= \sum_{j=1}^{d}(a_{ij} - z_{ij})^2 + \sum_{j=d+1}^{D} a_{ij}^2.\end{aligned}$$

The second term is fixed. We minimize the first term by choosing $z_{ij} = a_{ij}$ for every $j \le d$. Thus

$$\mathbf{z}_i^\star = U_d'(\mathbf{x}_i - \mathbf{b}), \qquad \widehat{\mathbf{x}}_i = \mathbf{b} + U_d U_d'(\mathbf{x}_i - \mathbf{b}).$$

The matrix $P_d = U_d U_d' \in \mathbb{R}^{D\times D}$ is the orthogonal projection onto the span of the retained directions. In particular,

$$P_d' = P_d, \qquad P_d^2 = U_d \underbrace{U_d'U_d}_{=I_d} U_d' = P_d.$$

The reconstruction residual $(I_D - P_d)(\mathbf{x}_i - \mathbf{b})$ is orthogonal to every column of U_d. Geometrically, $\widehat{\mathbf{x}}_i$ is the closest point to $\mathbf{x}_i$ in the affine subspace.

9.3.2 *Why we center the data*

Statement. For any fixed U_d, an optimal choice of $\mathbf{b}$ is the empirical mean

$$\bar{\mathbf{x}} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{x}_i.$$

Proof. Insert the optimal encoding into the objective. Since $\mathbf{x}_i - \mathbf{b} = (\mathbf{x}_i - \bar{\mathbf{x}}) + (\bar{\mathbf{x}} - \mathbf{b})$, we obtain

$$\begin{aligned} J &= \frac{1}{N}\sum_{i=1}^{N} \|(I_D - P_d)(\mathbf{x}_i - \mathbf{b})\|_2^2 \\ &= \frac{1}{N}\sum_{i=1}^{N} \|(I_D - P_d)(\mathbf{x}_i - \bar{\mathbf{x}})\|_2^2 \\ &\quad + \|(I_D - P_d)(\bar{\mathbf{x}} - \mathbf{b})\|_2^2. \end{aligned}$$

The cross term vanishes because $\sum_{i=1}^{N}(\mathbf{x}_i - \bar{\mathbf{x}}) = \mathbf{0}$. The last term is nonnegative and is zero at $\mathbf{b} = \bar{\mathbf{x}}$. More generally, shifting $\mathbf{b}$ along a direction in U_d gives the same affine subspace. Choosing the mean gives us a natural origin for the scores.

From here on, let $\widetilde{\mathbf{x}}_i = \mathbf{x}_i - \bar{\mathbf{x}}$. The PCA encoder and decoder are

$$f_{\text{enc}}(\mathbf{x}) = U_d'(\mathbf{x} - \bar{\mathbf{x}}), \qquad f_{\text{dec}}(\mathbf{z}) = \bar{\mathbf{x}} + U_d\mathbf{z}.$$

The encoding is a linear projection of the centered data. In the original coordinates, the reconstruction lies in a subspace shifted by $\bar{\mathbf{x}}$.

9.4 Choosing the principal directions

9.4.1 *Minimizing reconstruction error is maximizing variance*

Define the empirical covariance matrix

$$S = \frac{1}{N}\sum_{i=1}^{N} \widetilde{\mathbf{x}}_i\widetilde{\mathbf{x}}_i' = \frac{1}{N}X_c'X_c, \qquad X_c = X - \mathbf{1}_N\bar{\mathbf{x}}',$$

where $\mathbf{1}_N$ is the N-vector of ones. The diagonal entry $S_{\ell\ell}$ is the empirical variance of feature ℓ. For any unit vector $\mathbf{u}$, the variance of the data projected onto $\mathbf{u}$ is

$$\frac{1}{N}\sum_{i=1}^{N}(\mathbf{u}'\widetilde{\mathbf{x}}_i)^2 = \mathbf{u}'S\mathbf{u}.$$

We can use $1/(N-1)$ for the usual sample covariance when $N > 1$. Both conventions give the same principal directions and explained-variance ratios. We use $1/N$ here to match the normalization of J.

The projection and the residual are orthogonal. Therefore, by Pythagoras,

$$\|\widetilde{\mathbf{x}}_i\|_2^2 = \underbrace{\|U_d'\widetilde{\mathbf{x}}_i\|_2^2}_{\text{squared length retained}} + \underbrace{\|(I_D - P_d)\widetilde{\mathbf{x}}_i\|_2^2}_{\text{squared reconstruction error}}.$$

The first term is the squared length of the projected vector because multiplying by U_d preserves the length of a vector in $\mathbb{R}^d$. Averaging over i gives

$$J = \underbrace{\operatorname{tr}(S)}_{\text{total variance}} - \underbrace{\sum_{j=1}^{d}\mathbf{u}_j'S\mathbf{u}_j}_{\text{variance retained by } U_d}.$$

The total variance is fixed. To minimize J, we therefore choose the d orthonormal directions that retain the most variance.

9.4.2 Why we take the eigenvectors with the largest eigenvalues

Statement. An optimal U_d is given by the eigenvectors of S corresponding to its d largest eigenvalues.

Proof. The matrix $S = X_c'X_c/N$ is symmetric and positive semidefinite. By the spectral theorem we can write $S = Q\Lambda Q'$, where $Q = [\mathbf{q}_1, \ldots, \mathbf{q}_D]$ has orthonormal columns and

$$\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_D), \qquad \lambda_1 \geq \cdots \geq \lambda_D \geq 0.$$

For $d = 1$, write a unit vector $\mathbf{u} = \sum_{\ell=1}^{D} a_\ell \mathbf{q}_\ell$. Then

$$\mathbf{u}' S \mathbf{u} = \sum_{\ell=1}^{D} \lambda_\ell a_\ell^2 \leq \lambda_1 \sum_{\ell=1}^{D} a_\ell^2 = \lambda_1.$$

Taking $\mathbf{u} = \mathbf{q}_1$ attains the bound. So the first principal direction is the direction of maximum projected variance.

For general d, the total retained variance can be written as

$$\sum_{j=1}^{d} \mathbf{u}_j' S \mathbf{u}_j = \sum_{\ell=1}^{D} \lambda_\ell w_\ell, \qquad w_\ell = \sum_{j=1}^{d} (\mathbf{q}_\ell' \mathbf{u}_j)^2 = \|U_d' \mathbf{q}_\ell\|_2^2.$$

Each w_ℓ is between 0 and 1, since it is the squared length of the projection of a unit vector. The weights sum to d, because for each unit $\mathbf{u}_j$ the squared coefficients in the full basis Q sum to 1. Hence

$$\begin{aligned} \sum_{\ell=1}^{d} \lambda_\ell - \sum_{\ell=1}^{D} \lambda_\ell w_\ell &= \sum_{\ell=1}^{d} \lambda_\ell (1 - w_\ell) - \sum_{\ell=d+1}^{D} \lambda_\ell w_\ell \\ &\geq \lambda_d \left(\sum_{\ell=1}^{d} (1 - w_\ell) - \sum_{\ell=d+1}^{D} w_\ell \right) = 0. \end{aligned}$$

The inequality uses $\lambda_\ell \geq \lambda_d$ for $\ell \leq d$ and $\lambda_\ell \leq \lambda_d$ for $\ell > d$. Choosing $U_d = [\mathbf{q}_1, \ldots, \mathbf{q}_d]$ attains the bound, since then the first d weights are 1 and the rest are 0. Thus the optimal retained variance and the minimum reconstruction error are

$$\sum_{j=1}^{d} \lambda_j \qquad \text{and} \qquad J^\star = \sum_{j=d+1}^{D} \lambda_j.$$

Sidenote. A principal direction is a vector $\mathbf{q}_j$ in the original feature space. The corresponding *score* for observation i is $z_{ij} = \mathbf{q}_j' \widetilde{\mathbf{x}}_i$. Since the centered observations have mean zero, the scores do as well. Their empirical covariance is

$$\frac{1}{N} \sum_{i=1}^{N} \mathbf{z}_i \mathbf{z}_i' = U_d' S U_d = \operatorname{diag}(\lambda_1, \ldots, \lambda_d).$$

The scores are therefore uncorrelated, and the variance of the jth score is λ_j. The

sign of an eigenvector is arbitrary. Flipping its sign also flips its score, so their product in the reconstruction is the same. If eigenvalues tie, several orthonormal choices of directions can give the same optimal cost.

9.5 Computing PCA

9.5.1 Using the covariance matrix

1. Compute the empirical mean $\bar{\mathbf{x}}$ and center the data to obtain X_c.
2. Compute $S = X_c' X_c / N$ and find its eigenvalues and eigenvectors in decreasing order of eigenvalue.
3. Form U_d from the first d eigenvectors. The $N \times d$ matrix of scores is $Z = X_c U_d$, whose ith row is $\mathbf{z}_i'$.
4. To reconstruct in the original space, use $\widehat{X} = \mathbf{1}_N \bar{\mathbf{x}}' + Z U_d'$.

For a new observation, subtract the mean found during fitting and project onto the same U_d. When PCA is used before a supervised model, fit the mean and directions on the training data and use those quantities to transform the validation and test observations.

9.5.2 Using the singular value decomposition

As in the linear regression chapter, take the compact SVD. Here we write the left singular vectors as the columns of L to distinguish them from the PCA directions:

$$X_c = L \Gamma V', \qquad \Gamma = \operatorname{diag}(\gamma_1, \dots, \gamma_r), \qquad r = \operatorname{rank}(X_c).$$

Then

$$\begin{aligned} S &= \frac{1}{N} X_c' X_c \\ &= \frac{1}{N} V \Gamma \underbrace{L' L}_{= I_r} \Gamma V' = V \frac{\Gamma^2}{N} V'. \end{aligned}$$

Thus the right singular vectors of X_c are the eigenvectors of S with positive eigenvalues, and $\lambda_j = \gamma_j^2 / N$ for $j \le r$. For $d \le r$ we can take $U_d = V_d$ and

compute the scores directly as

$$Z = X_c V_d = L_d \Gamma_d.$$

This is especially useful when D is large, since we can work with X_c directly without forming the $D \times D$ covariance matrix.

Centering makes the rows of X_c sum to zero. Thus $r \leq \min\{N-1, D\}$. Directions beyond rank r have zero sample variance. Keeping the r directions with positive variance already reconstructs the training observations exactly.

9.5.3 Connection to low-rank approximation

The reconstruction of the centered data is

$$\widehat{X}_c = Z V_d' = L_d \Gamma_d V_d' = \sum_{j=1}^{d} \gamma_j \mathbf{l}_j \mathbf{v}_j'.$$

This is the truncated SVD. By the Eckart–Young theorem it is a best approximation to X_c among matrices of rank at most d, measured by the Frobenius norm. The connection with the PCA objective is

$$J^\star = \frac{1}{N} \| X_c - \widehat{X}_c \|_F^2 = \frac{1}{N} \sum_{j=d+1}^{r} \gamma_j^2 = \sum_{j=d+1}^{D} \lambda_j.$$

The first equality holds because the squared Frobenius norm adds up the squared lengths of the row residuals.

9.6 Explained variance and choosing the dimension

The fraction of variance explained by the first d principal components is

$$\mathrm{EV}(d) = \frac{\sum_{j=1}^{d} \lambda_j}{\sum_{j=1}^{D} \lambda_j} = 1 - \frac{J^\star}{\mathrm{tr}(S)}$$

when the total variance is positive. Thus if the first d components explain 95% of the variance, the average squared reconstruction error is 5% of the total variance.

One way to choose d is to take the smallest value that reaches a chosen threshold. For visualization, we might instead decide in advance to use $d = 2$ or $d = 3$.

A plot of λ_j against j shows how much variance each additional component contributes. A plot of $\mathrm{EV}(d)$ against d shows how quickly these contributions accumulate. If the eigenvalues fall quickly, a small number of components can capture most of the variation.

9.6.1 Worked example in two dimensions

Consider the four observations

$$(4,2), \quad (3,3), \quad (0,0), \quad (1,-1).$$

The mean is $(2,1)$, and the centered observations are $(2,1)$, $(1,2)$, $(-2,-1)$, and $(-1,-2)$. Their covariance is

$$S = \begin{bmatrix} 5/2 & 2 \\ 2 & 5/2 \end{bmatrix}.$$

The diagonal entries are equal, and the positive off-diagonal entry means the two features tend to move together. The eigenvectors and eigenvalues are

$$\mathbf{q}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \lambda_1 = \frac{9}{2}, \qquad \mathbf{q}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad \lambda_2 = \frac{1}{2}.$$

For $d = 1$ we keep $\mathbf{q}_1$. The first observation $\mathbf{x}_1 = (4,2)'$ has score

$$z_1 = \mathbf{q}_1'(\mathbf{x}_1 - \bar{\mathbf{x}}) = \frac{3}{\sqrt{2}}.$$

Its reconstruction is

$$\widehat{\mathbf{x}}_1 = \bar{\mathbf{x}} + z_1 \mathbf{q}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix} + \begin{bmatrix} 3/2 \\ 3/2 \end{bmatrix} = \begin{bmatrix} 7/2 \\ 5/2 \end{bmatrix}.$$

The residual is $(1/2, -1/2)'$, which is orthogonal to $\mathbf{q}_1$. The retained variance fraction is $(9/2)/5 = 0.9$, and the average squared reconstruction error is the

discarded eigenvalue $1/2$.

9.6.2 Example with faces

For the face images in the slides, $\bar{\mathbf{x}}$ is the *mean face*, obtained by averaging the intensity of each pixel across images. Each principal direction is itself a vector in pixel space, so we can reshape it into an image. These images are called *eigenfaces*. They represent directions of variation around the mean face.

A face is reconstructed as the mean face plus a weighted sum of the retained eigenfaces. The weights are the face's PCA scores. The slides compare reconstructions using all available directions and using 300, 200, or 100 eigenfaces. Using fewer directions requires fewer scores per image, while the discarded directions account for the additional reconstruction error.

9.7 PCA and preprocessing

9.7.1 Centering and scaling

Centering subtracts a mean from each feature. Standardization also divides each feature by its empirical standard deviation. For feature ℓ with standard deviation $s_\ell > 0$, the standardized value is

$$x_{i\ell}^{\text{std}} = \frac{x_{i\ell} - \bar{x}_\ell}{s_\ell}.$$

This matters because PCA depends on the scale of the features. For example, measuring a length in centimeters instead of meters multiplies that feature's variance by 100^2. If the choice of units is arbitrary, standardizing can prevent it from dominating the principal directions. PCA on standardized features is eigendecomposition of their correlation matrix.

Standardization sets the variance of each original coordinate to one. There may still be correlations between coordinates, and some linear combinations may still have much larger variance than others.

9.7.2 PCA whitening and ZCA

PCA gives centered coordinates whose covariance is diagonal. To *whiten* these coordinates, divide each score by its standard deviation $\sqrt{\lambda_j}$. Let $\Lambda_d = \text{diag}(\lambda_1, \ldots, \lambda_d)$ and suppose the retained eigenvalues are positive. The whitened representation is

$$\mathbf{w} = \Lambda_d^{-1/2} U_d'(\mathbf{x} - \bar{\mathbf{x}}).$$

Its empirical covariance is

$$\Lambda_d^{-1/2} \underbrace{U_d' S U_d}_{=\Lambda_d} \Lambda_d^{-1/2} = I_d.$$

Thus the retained coordinates are uncorrelated and have unit variance. The positivity condition matters because a coordinate with zero variance cannot be divided by its standard deviation.

In the full-rank case, *ZCA whitening* rotates the whitened scores back to the original coordinate axes:

$$\mathbf{w}_{\text{ZCA}} = U \Lambda^{-1/2} U'(\mathbf{x} - \bar{\mathbf{x}}).$$

Here U is the full orthogonal eigenvector matrix of S. The covariance is $U I_D U' = I_D$. With rank-deficient data we can whiten the positive-variance directions in their lower-dimensional score space.

Sidenote. The order matters if we want to reduce dimension and whiten. First use the eigenvalues to decide which directions to keep. Then whiten those retained scores if we want them on the same scale. If we whiten all directions first, the resulting covariance is the identity and every unit direction has the same variance.

9.8 When does PCA work well?

PCA works well as a compression method when the data lie near a low-dimensional affine subspace. Then most variance is captured by a few directions and the discarded eigenvalues are small. If the observations instead lie along

a strongly curved shape, one straight line can give a large reconstruction error even if the shape can be described by a single parameter. This motivates the nonlinear dimensionality reduction methods discussed later.

Large explained variance can also be a poor guide to usefulness for prediction. For example, suppose the four possible inputs $(\pm 10, \pm 1)$ occur equally often and the label is the sign of the second coordinate. The covariance matrix is $\mathrm{diag}(100, 1)$. One-dimensional PCA keeps the first coordinate and explains $100/101$ of the input variance. But each value of the first coordinate occurs with both labels. All the label information is in the discarded coordinate. PCA chooses directions using the variation of the inputs, so whether those directions are useful for predicting a label depends on how the label relates to the inputs.

10 Stochastic Neighborhood Embedding (non-linear Dimensionality Reduction)

10.1 The task of dimensionality reduction

Recall from PCA that we have N observations $\mathbf{x}_i \in \mathbb{R}^D$ and would like to represent each observation using $d < D$ numbers. Here I write X for the dataset and Y for its embedding. Formally we want

$$X = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\} \subset \mathbb{R}^D \qquad \longrightarrow \qquad Y = \{\mathbf{y}_1, \ldots, \mathbf{y}_N\} \subset \mathbb{R}^d,$$

where $\mathbf{y}_i$ is the new position of observation $\mathbf{x}_i$. The number of observations is still N. It is the number of coordinates per observation that we are reducing. For visualization we usually choose $d = 2$ or $d = 3$ so that we can plot the points.

Recall also the previous section on clustering. There are two slightly different things we might want to do. We can reduce the data first and then apply a clustering algorithm to the reduced features. Or we can cluster the original data and use a low-dimensional plot to see where the cluster members end up. In either case the question is what structure we want the embedding to preserve. For example, keeping nearby observations together is a different objective from minimizing reconstruction error (as we did in PCA).

10.1.1 Why might a linear projection be a problem?

PCA works well if the data lie close to a low-dimensional plane (or the higher-dimensional equivalent of a plane). But what if the data lie on a curved surface? Consider the Swiss roll on slides 7–8. Imagine a sheet of paper rolled up in 3D. We need three coordinates to write down the position of a point on the roll, but if we unroll the paper we only need two coordinates to locate that same point. I.e. the *ambient dimension* is $D = 3$ and the *intrinsic dimension* is $d = 2$.

Now imagine projecting the roll onto a plane. Points on different layers of the roll can end up on top of each other, even if we would have to walk a long way along the paper to get from one to the other. This is the issue with using a single linear projection. What we would like to do in this example is something more like unrolling the paper.

The *manifold assumption* is that the data lie on (or close to) a low-dimensional surface inside $\mathbb{R}^D$. A d-dimensional manifold locally looks like $\mathbb{R}^d$. By locally I mean that if we zoom in on a small patch of the Swiss roll it looks like a piece of a plane, even though the whole thing is curved. Manifold learning tries to recover or approximate this underlying structure.

Another example from the slides is ethanol. The molecule has 9 atoms and we can describe one configuration using $D = 9 \cdot 3 = 27$ coordinates. However, much of the variation in the configurations can be described by two rotation angles. Each angle wraps around, which gives the approximate torus (doughnut shape) on slide 10. So there can be a big difference between how many numbers we use to record the data and how many numbers we need to describe the main ways it varies.

10.2 Symmetric Stochastic Neighbor Embedding (SNE)

The idea of SNE is to turn distances into probabilities. Start at $\mathbf{x}_i$ and imagine choosing another observation as its neighbor at random. Nearby observations should be more likely to be chosen. Then do something similar for the points in the plot and move them around until the two sets of probabilities agree as well as possible.

10.2.1 *Affinities in the original data X*

Formally let $p_{j|i}$ be the probability of choosing $\mathbf{x}_j$ as a neighbor when starting from $\mathbf{x}_i$. For $j \neq i$ define

$$p_{j|i} = \frac{\exp\left\{-\|\mathbf{x}_i - \mathbf{x}_j\|_2^2/(2\sigma_i^2)\right\}}{\sum_{k\neq i}\exp\left\{-\|\mathbf{x}_i - \mathbf{x}_k\|_2^2/(2\sigma_i^2)\right\}}, \qquad p_{i|i} = 0.$$

The denominator just makes the probabilities sum to 1 for each starting point i. The role of σ_i is to decide how quickly the weight falls as we move away from $\mathbf{x}_i$. If σ_i is very small then almost all the probability goes to the closest point (or points if there is a tie). If σ_i is very large then all other observations are about equally likely to be chosen.

Why do we call it *symmetric* SNE? Notice that $p_{j|i}$ and $p_{i|j}$ can differ even though the distance is the same in both directions. For example, $\mathbf{x}_j$ might be one of the closest points to $\mathbf{x}_i$, while $\mathbf{x}_j$ itself has lots of even closer neighbors. To get one symmetric affinity for the pair we use

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}.$$

Thus $p_{ij} = p_{ji}$ and $\sum_{i,j} p_{ij} = 1$. We write $P = (p_{ij})$ for this probability distribution over pairs. Once we have calculated P, it contains everything we need from X for the optimization. So we could also start with a matrix of pairwise dissimilarities if that is how the data are given.

10.2.2 *How do we choose σ_i?*

The idea is to choose approximately how many neighbors each point should pay attention to. This is called the *perplexity*. Let P_i denote the distribution with probabilities $p_{j|i}$. Then

$$H(P_i) = -\sum_{j\neq i} p_{j|i}\log_2(p_{j|i}), \qquad \text{perp}(P_i) = 2^{H(P_i)}.$$

To see what this means, imagine that 8 neighbors each get probability $1/8$ and the rest get zero. Then $H(P_i) = \log_2(8) = 3$ and the perplexity is $2^3 = 8$. In general the probabilities are unequal, so we can think of perplexity as an *effective* number of neighbors. For example a perplexity of 30 still gives different weights to different neighbors.

We choose one target perplexity (say 30) and then find a separate σ_i for each observation. Increasing σ_i spreads the probabilities out and increases the perplexity, so we can find the right value by binary search. Intuitively, if $\mathbf{x}_i$ is in a dense region we only need to look a short distance to find about 30 neighbors. In a sparse region we need to look further. Thus for a fixed perplexity the bandwidth will typically be smaller in dense regions and larger in sparse regions.

10.2.3 Affinities in the embedding Y and the objective

Now let $\ell_{ij} = \|\mathbf{y}_i - \mathbf{y}_j\|_2$ be the distance between the two observations in the embedding. Symmetric SNE uses the Gaussian-like weight $\exp\{-\ell_{ij}^2\}$ and normalizes it over all distinct pairs

$$q_{ij} = \frac{\exp\{-\ell_{ij}^2\}}{\sum_{k=1}^{N}\sum_{l\neq k}\exp\{-\ell_{kl}^2\}}, \qquad i \neq j, \qquad q_{ii} = 0.$$

Write $Q(Y) = (q_{ij})$. The important difference here is that P is already known from the original data, while $Q(Y)$ changes every time we move the points $\mathbf{y}_i$.

How do we decide whether the points are in the right places? We measure the mismatch between P and $Q(Y)$ using the KL divergence

$$C(Y) = \mathrm{KL}(P\|Q(Y)) = \sum_{i=1}^{N}\sum_{j\neq i} p_{ij}\log\left(\frac{p_{ij}}{q_{ij}}\right), \qquad \min_{\mathbf{y}_1,\ldots,\mathbf{y}_N\in\mathbb{R}^d} C(Y).$$

If two points are close in X (large p_{ij}) but end up far apart in Y (small q_{ij}), we pay a large penalty. A pair with small p_{ij} gets less weight in the sum. This is why the objective puts so much emphasis on keeping the original neighbors together.

The gradient also has a nice interpretation. In symmetric SNE it is

$$\nabla_{\mathbf{y}_i} C = 4 \sum_{j \neq i} (p_{ij} - q_{ij})(\mathbf{y}_i - \mathbf{y}_j).$$

Remember that gradient descent moves in the *negative* gradient direction. So if $p_{ij} > q_{ij}$ the pair contributes a pull towards each other, and if $p_{ij} < q_{ij}$ it contributes a push apart. The actual movement of $\mathbf{y}_i$ is the sum of these contributions from all the other points.

Basically we compute P once, start the $\mathbf{y}_i$ at small random coordinates and then alternate between calculating $Q(Y)$ and taking a gradient step. We can add momentum just like in the neural network section (remember the heavy ball). Here we are updating the coordinates themselves. This is why the method is called *non-parametric*. PCA gives us a fitted projection that can be applied directly to a new observation, whereas the SNE procedure above gives us coordinates for the observations we fitted it to. The objective is non-convex, so different starting positions can give different results.

10.3 t-SNE

10.3.1 The crowding problem

Why change symmetric SNE? Imagine a point at the origin in 2D with neighbors at $(1, 0)$, $(-1, 0)$, $(0, 1)$ and $(0, -1)$. All four neighbors are distance 1 from the origin, and they have room to be separated from each other. Now try to arrange them on a line. There are only two positions at distance 1 from the origin! So we have to put some neighbors on top of each other or change some of the distances. This is the *crowding problem*. There are fewer directions to spread out in when we go down in dimension.

With the Gaussian weights in Y, affinities fall very quickly as we move points apart. So even pairs that are only moderately similar can end up competing for space close together in the embedding. The idea of t-SNE is to give such points more room by using a weight that falls more slowly with distance.

10.3.2 What changes in t-SNE?

We calculate the same P from the original data and minimize the same KL divergence. The change is how we calculate $Q(Y)$. The new weights are

$$q_{ij} = \frac{(1 + \ell_{ij}^2)^{-1}}{\sum_{k=1}^{N} \sum_{l \neq k} (1 + \ell_{kl}^2)^{-1}}, \qquad i \neq j, \qquad q_{ii} = 0.$$

The weight $(1 + \ell_{ij}^2)^{-1}$ comes from the Student t-distribution with one degree of freedom (also called the Cauchy distribution). Hence the name *t-distributed Stochastic Neighbor Embedding*.

What does the heavier tail actually do? At distance $\ell_{ij} = 3$ the old weight is $e^{-9} \approx 0.00012$ and the new weight is $1/(1 + 9) = 0.1$ (both before normalization). I.e. the new weight can still be appreciable at distances where the Gaussian weight is already very close to zero. This lets the embedding put moderately similar points further apart while still assigning them some affinity. Close neighbors can stay close and different groups get more room to spread out. The optimization is again gradient descent with momentum, with the gradient now calculated using the new $Q(Y)$.

10.4 What do the plots in the slides show?

10.4.1 MNIST

Each observation is a 28×28 grayscale picture of a handwritten digit, so $D = 784$ and $N = 70{,}000$. In the example on slides 27 and 35–36, PCA first reduces the data from 784 to 50 dimensions. Then SNE or t-SNE takes those 50 coordinates down to 2 (with perplexity 50 in this example). So PCA and t-SNE can actually be used in the same pipeline.

The symmetric SNE plot already shows some structure, but several of the digit groups run into each other. In the t-SNE plot the groups are easier to tell apart. Note that the digit labels are used to color and annotate the plot afterwards. The embedding itself is unsupervised. Each dot is an image, which is easier to see on slide 40 where some of the digits are drawn at their positions in the embedding.

10.4.2 Gaussian mixture, Swiss roll and spiral

- **Gaussian mixture (slide 37).** The data come from four Gaussian distributions with the same covariance matrix. The PCA plot already shows the four groups and the t-SNE plot separates them more clearly in this example. The gaps in the t-SNE plot describe the layout of the embedding. To compare how far apart the Gaussians actually are we would go back to the original data.
- **Swiss roll (slide 38).** The PCA projection still looks like a roll viewed from the side. The t-SNE plot keeps many of the nearby colors together but breaks the roll into separate pieces. So even when t-SNE does a good job with local neighbors, it can change how the whole surface is connected.
- **Spiral (slide 39).** The PCA projection still shows the oscillations of the two spirals and makes them cross in the plot. t-SNE lays them out as two nearly parallel curves. The interesting thing here is which points stay next to each other along each curve. The shape itself changes quite a lot.

10.4.3 The home assignment example

The original data on slide 41 look like two linked rings in 3D. The two PCA plots on slide 43 show how the projection changes when we normalize the input. In the t-SNE plots the two rings are visually separated for both perplexities. At perplexity 30 the rings look rougher and broken up in places, while at perplexity 400 they look much smoother. This is a good example of what changing the perplexity does. With 400 the affinities around each point are spread over a much larger neighborhood, so the algorithm is trying to preserve similarities over a broader part of the data.

10.5 How to read a t-SNE plot

The first thing I would look at is which observations end up near each other. The axes can be rotated or translated without changing any of the pairwise distances, so the axis values themselves have no particular meaning. The sizes of groups and the gaps between them can also change with the perplexity and the optimization. For example a large empty gap might come from the way t-SNE arranged the points in 2D. The Swiss roll is a nice reminder of this since

we know it started out as one continuous sheet.

Recall that clustering gives us a group assignment for each point, while t-SNE gives us a position for each point. To use them together as on slide 45 we can run e.g. k-means on X, fit a 2D t-SNE embedding and then color each $\mathbf{y}_i$ by the cluster assigned to $\mathbf{x}_i$. This lets us see how the cluster assignments line up with the neighborhoods in the plot.

One practical detail is that the exact method works with all pairs of observations. So for a fixed output dimension the work per iteration grows like N^2. The MNIST example uses an approximate implementation, which makes it possible to use the method on a dataset of that size.